R&T Semestre 2 2025/2026

SAE22 : Mesurer et caractériser un signal

Simulations numériques

Cette Situation Acquisition et d'Evaluation va traiter de réalisation ou acquisition de signaux numériques et de leurs exploitations. Elle se décompose en 4 parties :

- Partie 1 : Prise en main de Matlab
- Partie 2 : Calcul numérique d'une intégrale
- Partie 3 : Etude d'un signal temporel
- Partie 4 : Etude d'un canal Radio

Pour réaliser le travail demandé, vous disposez de :

- 8 séances de 2h encadrées
- 4 séances de 2h en autonomie à réaliser en salle à l'IUT.
- Du travail personnel à réaliser entre les séances.

L'évaluation se fera en contrôle continu sur la progression et vous aurez à produire :

- Un dossier de fichiers zip comprenant vos programmes commentés
- Un compte-rendu répondant aux différentes questions et analyses demandées.

Partie 1 : Prise en main de Matlab

I / Formation Matlab : « Onramp »

- Suivre les chapitres 1 (course Overview) à 9 (Plotting data) du module « Onramp » d'autoformation de Matlab : https://matlabacademy.mathworks.com/

II / Génération d'un signal sinusoïdal x(t)

On désire générer et afficher le signal $x(t) = 5 \times cos(2 \times \pi \times 1000 \times t)$ où x(t) est une tension en Volts.

Matlab étant un logiciel numérique, il va devoir **échantillonner** ce signal. On choisira ***fe*=40kHz** comme fréquence d'échantillonnage.

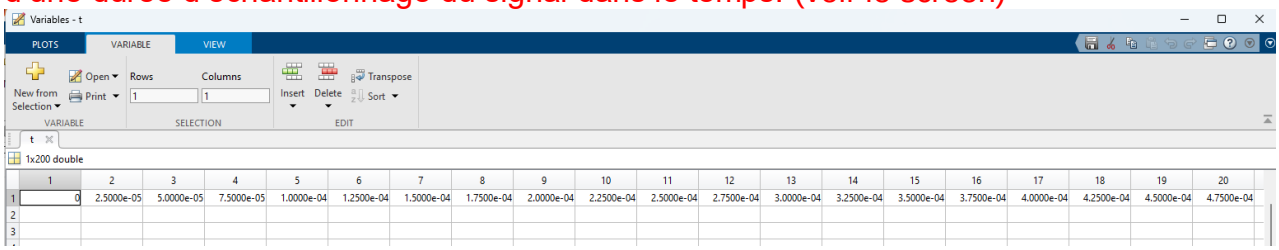
Le début du programme Matlab est le suivant :

```
1 -    f=1000;  %fréquence du signal sinusoïdal
2 -    A=5;  %amplitude de la sinusoïde
3
4 -    fe=40000; %fréquence d'échantillonnage
5 -    Te=1/fe;  %durée d'un échantillon
6 -    Ns=200; %nombre d'échantillons
7
8 -    t=0:Te:(Ns-1)*Te;
9
10 -   x=5*cos(2*pi*f*t);
```

Figure 1: début du programme de génération d'un cosinus

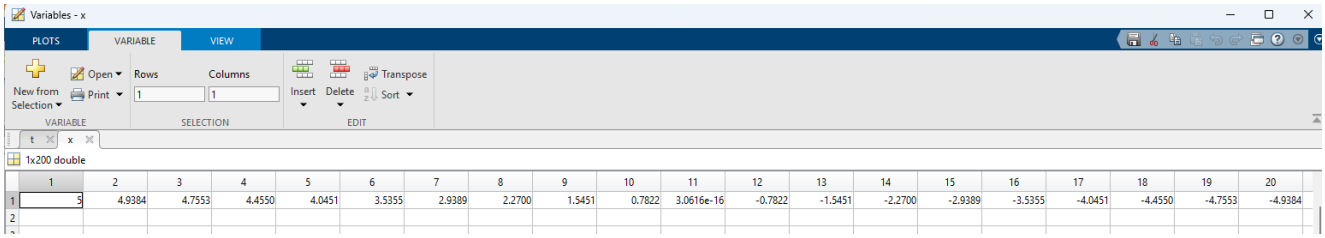
1) Que représente la variable t ? Quelle est sa nature ?

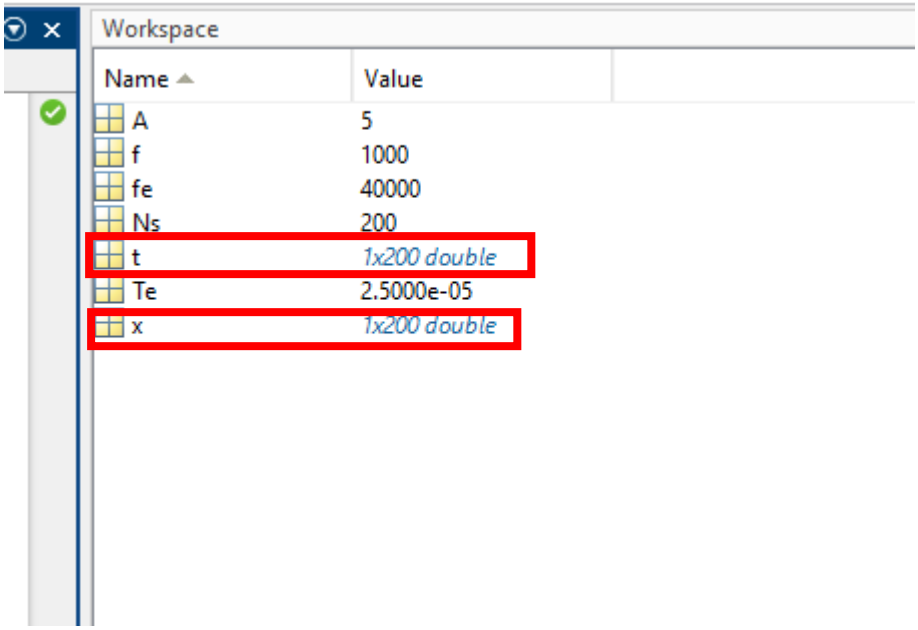
La variable t représente le vecteur temps. Sa nature est un tableau contenant les points d'échantillonnage du signal qui vont de 0 à (Ns-1)\Te par pas de Te. Et Te est la période d'une durée d'échantillonnage du signal dans le temps. (voir le screen)



2) Que fait la ligne 10 ? Quelle est la nature de x ?

La ligne 10 représente le signal x(t)=5cos(2pi1000t) d'amplitude 5V et de fréquence f = 1000 Hz pour chaque instant contenu dans le vecteur de temps t. La variable x est de même nature que t c'est un vecteur de même dimension qui contient les valeurs d'amplitude du signal. (voir le screen)





t et x ont une dimension 1x200 c'est-à-dire 1 ligne et 200 colonnes.

3) Combien de période du signal $x(t)$ seront calculées ?

La fréquence du signal est f=1000, ce qui donne une période T = 1/f = 0.001. La durée totale générée est égale à Ns\Te. Avec Ns=200 et Te=1/40000, la durée est de 200/40000 = 0.005. Le nombre de périodes calculées est donc la durée totale divisée par la période : 0.005 / 0.001 = 5 périodes.

La période du signal correspond au temps pour faire un cycle complet donc 0,001 seconde pour une fréquence de 1000 Hz pour notre signal. Par contre la durée totale du signal est échantillonnée par le nombre total d'échantillons (200), ce à quoi on le multiplie par l'intervalle de temps entre chacun d'eux cad que la période échantillonnée est égale à Te = 1/fe = 2.5e^05, ce qui donne 2.5e^05 seconde.

Mais il faut faire Ns*Te parce qu'on veut reconstituer les intervalles échantillonnés en une durée de temps complet, car Te est une période d'un seul échantillon alors qu'il y en a 200 au total donc pour reconstituer toutes les périodes on multiplie le nombre d'échantillon total sur la période échantillonnée. Puis on divise simplement cette durée totale par le temps que prend la période T du signal, et on calcule 0.005 / 0.001 = 5 périodes, et donc par ce calcul on a reconstitué 5 périodes complètes.

4) Quelle petite critique ou amélioration peut être apportée à ce programme ?

L'amélioration consiste à utiliser la variable A qui a été intelligemment définie à la ligne 2 (A = 5;). La ligne 10 devrait s'écrire : x=A*cos(2*pi*f*t); . Ce qui permet plus tard de changer l'amplitude et les valeurs aussi plus facilement si le code venait à être beaucoup plus long et plus complexe.

- **Faire la saisie de ce programme** (Il est recommandé de travailler avec des « live script » qui est une forme de programmation qui permet de commenter et de mettre en forme le programme. De cette façon le compte-rendu de cette SAE sera partiellement fait). Nommer par exemple ce programme : P1sinus.mlx.

III / Affichage du signal x(t)

- Chercher de l'aide sur la fonction « plot » puis compléter le programme suivant afin d'afficher x(t) avec les informations suivantes :

  - Couleur : rouge
  - Mettre un titre : title
  - Mettre une légende : legend
  - Mettre le nom des axes sur la figure : xlabel et ylabel
  - Fixer les axes suivants : de 0 à 2ms et de -10V à +10V : axis

Il faut obtenir une figure similaire à la Figure 2 :

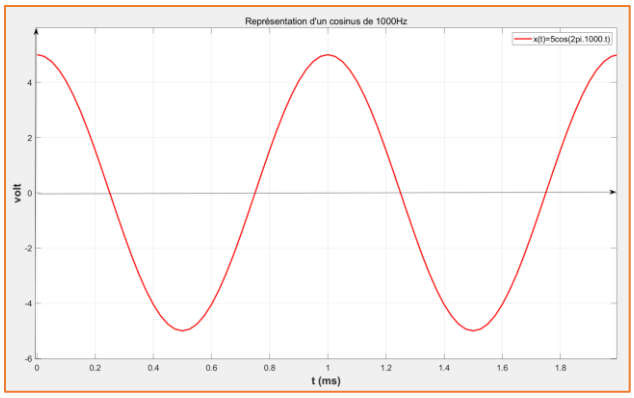


Figure 2 : affichage de la fonction cosinus x(t)

On veut maintenant créer un signal $y(t) = 3 \times sin(2 \times \pi \times 1000 \times t)$

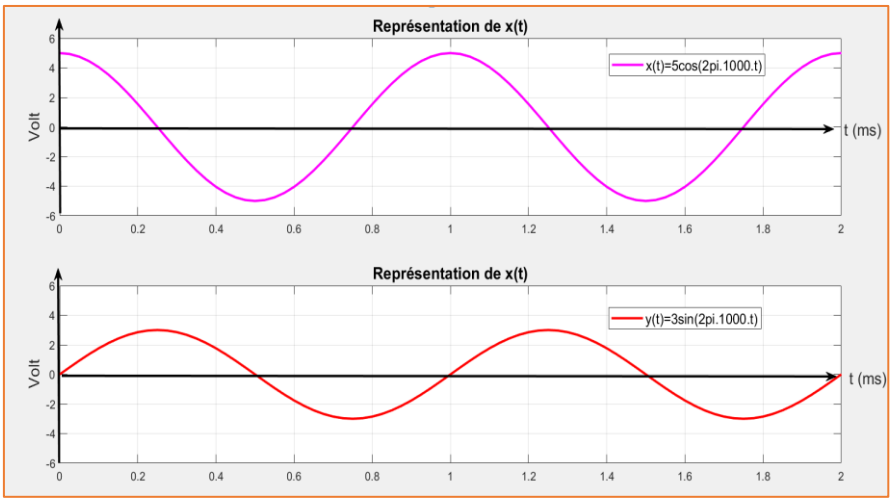
- Compléter le programme afin de créer et d'afficher le signal $y(t)$ en bleu.

```
title('Représentation Figure 2')
plot(t*1000, x, 'r')
xlabel('t (ms)')
ylabel('Volt')
axis([0 2 -10 10])
grid on
```

- Etudier la fonction « subplot » au moyen du menu « Help ».

subplot(2, 1, 1) % la fonction subplot sert à afficher plusieurs graphiques séparément sur une même image lorsque le code est lancé, la 1ère valeur représente la ligne, la 2ème valeur les colonnes et la dernière valeur c'est l'ordre de la position du graphique si c'est 1 ca apparaitra en 1er.

✋ Modifier le programme afin d'afficher les deux courbes sur deux graphes sur la même figure et obtenir la représentation suivante :



Le code à mettre est le suivant

```
13  subplot(2, 1, 1)
14  plot(t * 1000, x, 'b')
15  title('Représentation de x(t)')
16  xlabel('t (ms)')
17  ylabel('Volt')
18  axis([0 2 -6 6])
19  grid on
20
21
22  subplot(2, 1, 2)
23  plot(t * 1000, y, 'r')
24  title('Représentation de y(t)')
25  xlabel('t (ms)')
26  ylabel('Volt')
27  axis([0 2 -6 6])
28  grid on
```

Figure 3 : représentation de deux signaux en utilisant la commande « subplot »

Il est parfois intéressant d'avoir les 2 représentations graphiques sur le même graphique.

Rechercher comment afficher 2 courbes sur le même graphique avec la fonction « plot ».

plot(t * 1000, x, 'b', t * 1000, y, 'r') % x(t) en bleu ('b') et y(t) en rouge('r')

✋ Compléter le programme pour obtenir la représentation suivante :

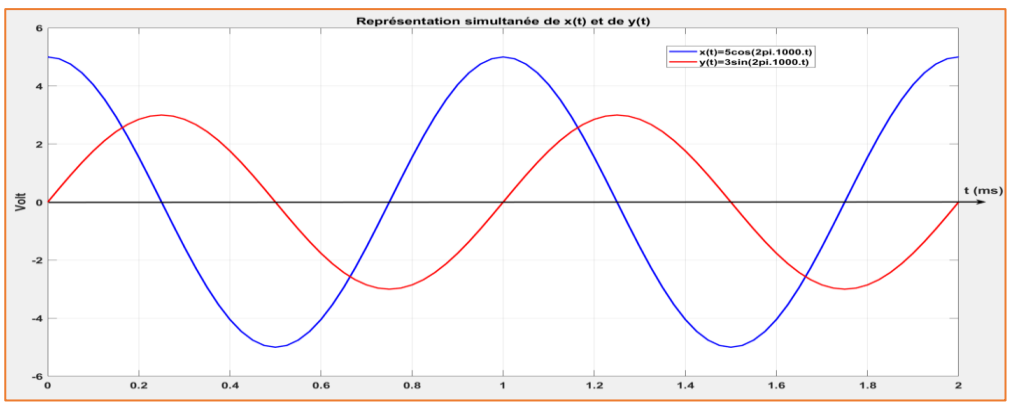


Figure 4 : représentation de deux signaux sur un même graphique

Le code a mettre est le suivant :

```
12
13      plot(t * 1000, x, 'b', t * 1000, y, 'r') % x(t) en bleu ('b') et y(t) en rouge ('r')
14
15
16      title('Représentation simultanée de x(t) et de y(t)')
17      xlabel('t (ms)')
18      ylabel('Volt')
19      legend('x(t)=5cos(2pi.1000.t)', 'y(t)=3sin(2pi.1000.t)')
20
21
22      axis([0 2 -6 6])
23      grid on
```

IV / Formation Matlab : suite

- Finir l'autoformation à Matlab Onramp.





MathWorks | Training Services

Course Completion Certificate

Amine ADHAME

has successfully completed **100%** of the self-paced training course

MATLAB Onramp

DIRECTOR, TRAINING SERVICES

26 May 2026

V / Affichage du spectre

Matlab dispose d'une fonction FFT (Fast Fourier Transform) permettant de calculer la transformée de Fourier d'un signal.

Afin de faciliter le travail, une fonction, « spectre.m », permettant d'obtenir le spectre d'un signal, est fournie.

✋ Télécharger sur l'ENT puis décompresser le dossier « TP initiation Matlab.zip » contenant :

- Le programme principal : « affichage_spectre_sinus.m »
- Une fonction pour le calcul des spectres : « spectre.m »

1) Etudier la ligne 28 ainsi que la fonction « spectre.m » à laquelle elle fait appel.

```
27      %calcul de X(f): spectre en amplitude de x(t)
28 -    [X f]=spectre(x,fe,Ns);  % appel à la fonction  spectre.
```

La ligne 28 appelle la fonction spectre.m pour calculer le contenu fréquentiel du signal x à partir de sa fréquence d'échantillonnage fe. Elle génère en sortie le spectre en amplitude X et le vecteur de fréquences f associé

2) Exécuter le script et valider le résultat.

Remarque : par la suite vous pourrez réutiliser cette fonction « spectre.m » pour tracer le spectre de n'importe quel signal.

- Créer une fonction « spectredBV.m » afin d'afficher maintenant le spectre en dBV ou dBµV. On utilisera pour cela la fonction $log10(X)$. On ne calculera pas ici la valeur efficace pour le moment.

```
function [X_dBV] = spectredBV(X)
    X_dBV = 20 * log10(X);
end
```

- Compléter le programme pour tracer le spectre en dBV.

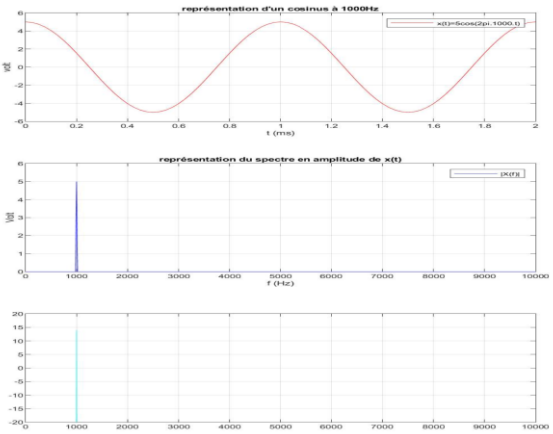
```
[X, f_axe] = spectre(x, fe, Ns);

X_dBV = spectreDBV(X);
```

- Valider le résultat obtenu.

Oui c'est bon

- Réaliser l'affichage comme montré ci-dessous, en mieux (penser aux noms des axes et autres …)



Code a mettre :

```
subplot(3, 1, 2)
plot(f_axe, X, 'b') % 'b' pour bleu
title('ReprÃ©sentation du spectre en amplitude de x(t)')
xlabel('FrÃ©quence f (Hz)')
ylabel('Amplitude (V)')
legend('|X(f)|')
axis([0 10000 0 6]) % On limite l'axe X pour bien voir la raie Ã  1000 Hz
grid on


subplot(3, 1, 3)
plot(f_axe, X_dBV, 'c') % 'c' pour cyan (bleu clair comme sur ta capture)
title('ReprÃ©sentation du spectre de x(t) en dBV')
xlabel('FrÃ©quence f (Hz)')
ylabel('Amplitude (dBV)')
legend('|X(f)|_{dBV}')
axis([0 10000 -20 20]) % Ajuste ces limites selon ce que renvoie ton log10
grid on
```

VI / Multiplication de deux signaux sinusoïdaux

On veut à présent réaliser la multiplication de deux signaux sinusoïdaux. En effet cette étape est fondamentale dans la transmission de signaux car elle réalise une transposition de fréquence. Ceci sera vu à l'aide du spectre de ce produit.

Les caractéristiques de l'échantillonnage seront les suivantes :

- Fréquence d'échantillonnage : *fech*=40kHz
- Nombre d'échantillons : Ns=2000

Les signaux à multiplier sont les suivants :

- $x1(t) = A1 \times cos(2 \cdot \pi \cdot f_1 \cdot t)$ avec A1=5V et *f1*=1kHz
- $x2(t) = A2 \times cos(2 \cdot \pi \cdot f_2 \cdot t)$ avec A2=3V et *f2*=3kHz

✋ Générer les deux signaux $x1(t)$ et $x2(t)$.



1) Vérifier la taille des vecteurs $x1$ et x2 créés. Justifier.

On voit bien dans la capture que les vecteurs x1 et x2 font 2000 colonnes

2) Quelle est la différence entre les opérations suivantes sous Matlab : « ***** » et « **.*** » ? Quel est l'opérateur à utiliser pour la multiplication demandée ?

L'opérateur * : C'est la multiplication matricielle.

L'opérateur .* : C'est la multiplication point par point. Elle multiplie chaque élément du premier vecteur par l'élément correspondant du second.

Pour multiplier deux signaux temporels, on utilise .* car on souhaite multiplier les signaux à chaque instant t

3) Quelle est l'opération réalisée avec l'opérateur « .* » ? C'est-à-dire, en fonction des indices, quels sont les termes multipliés entre eux ? (z(1)=x1(??)*x2(??) ….)

z(1) = x1(1) * x2(1)

z(2) = x1(2) * x2(2)

4) Calculer *z* le produit de ces deux signaux : $\boldsymbol{z = x1 .* x2}$

z = x1 .* x2;

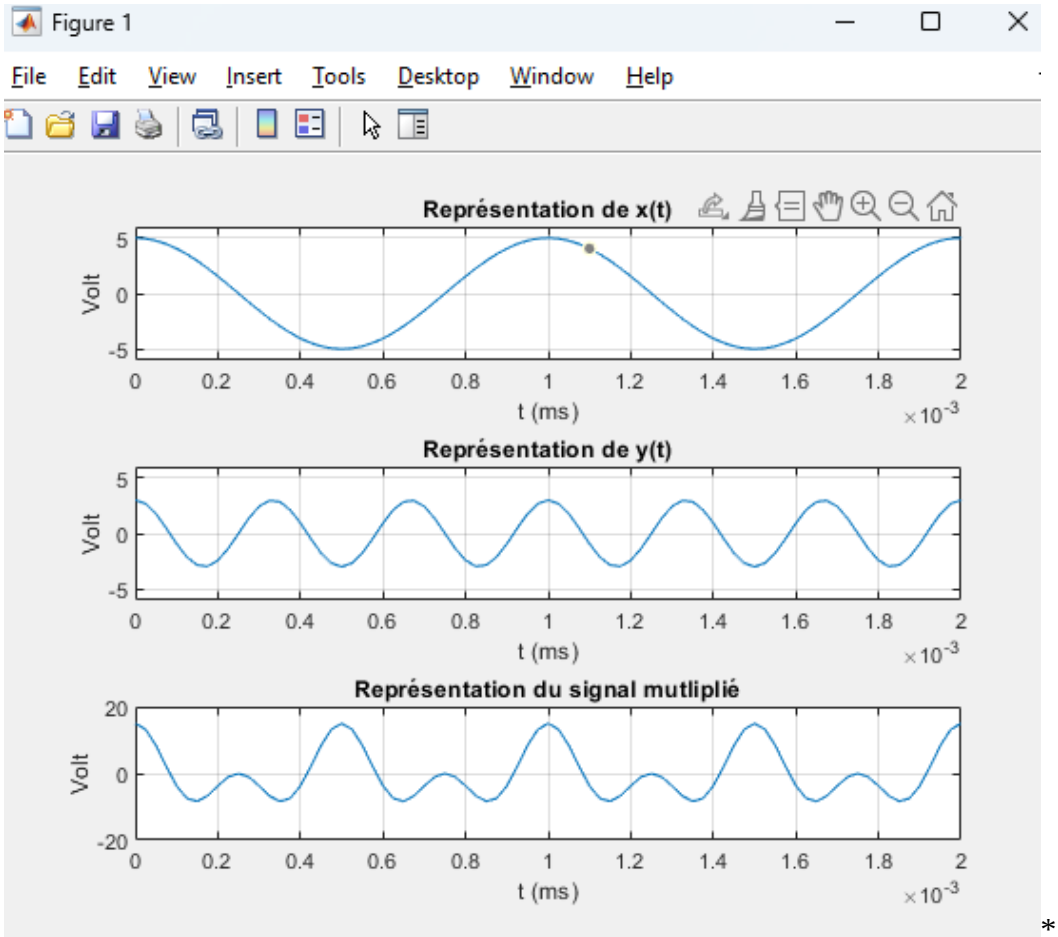
5) Quelle est la dimension du vecteur z ? Justifier.

La dimension du vecteur est de 1*2000 car l'opérateur .* n'est pas un calcul matriciel.

- Afficher sur une même figure, mais sur trois graphes les chronogrammes de $x1(t), x2(t)\ et\ z(t)$.

```
%representation de x1
subplot(3,1,1);
plot(t, x1);
title('Représentation de x(t)');
ylabel('Volt'); grid on;
xlabel('t (ms)'); grid on;
xlim([0 0.002])
ylim([-6 6])
%representation de x2
subplot(3,1,2);
plot(t, x2);
title('Représentation de y(t)');
ylabel('Volt'); grid on;
xlabel('t (ms)'); grid on;
xlim([0 0.002])
ylim([-6 6])
%representation du signal multiplié
subplot(3,1,3);
plot(t, z);
title('Représentation du signal mutliplié');
ylabel('Volt'); grid on;
xlabel('t (ms)'); grid on;
xlim([0 0.002])
ylim([-20 20])
```

Ce code nous permet de nous donner le signal en représentation temporelle. Lorsqu'on l'exécute on à ce résultat



*

✋ Calculer les spectres de ces trois signaux et afficher les sur trois graphes d'une nouvelle figure.

Pour ça il faut mettre la fonction spectre.m qui fait la fft

Le code de l'affichage ressemble finalement a ça :

```
% Signal x1
% Représentation temporelle
subplot(3,2,1);
plot(t * 1000, x1);
title('Représentation de x(t)');
ylabel('Volt'); grid on;
xlabel('t (ms)'); grid on;
xlim([0 2]);
ylim([-6 6]);

% Représentation fréquentielle
subplot(3,2,2); % Corrigé: un seul "subplot"
plot(f, specX);
title('Spectre en amplitude du signal x(t)');
ylabel('dBV'); grid on;
xlabel('f (Hz)'); grid on;
xlim([0 10000]);
ylim([0 20]); %

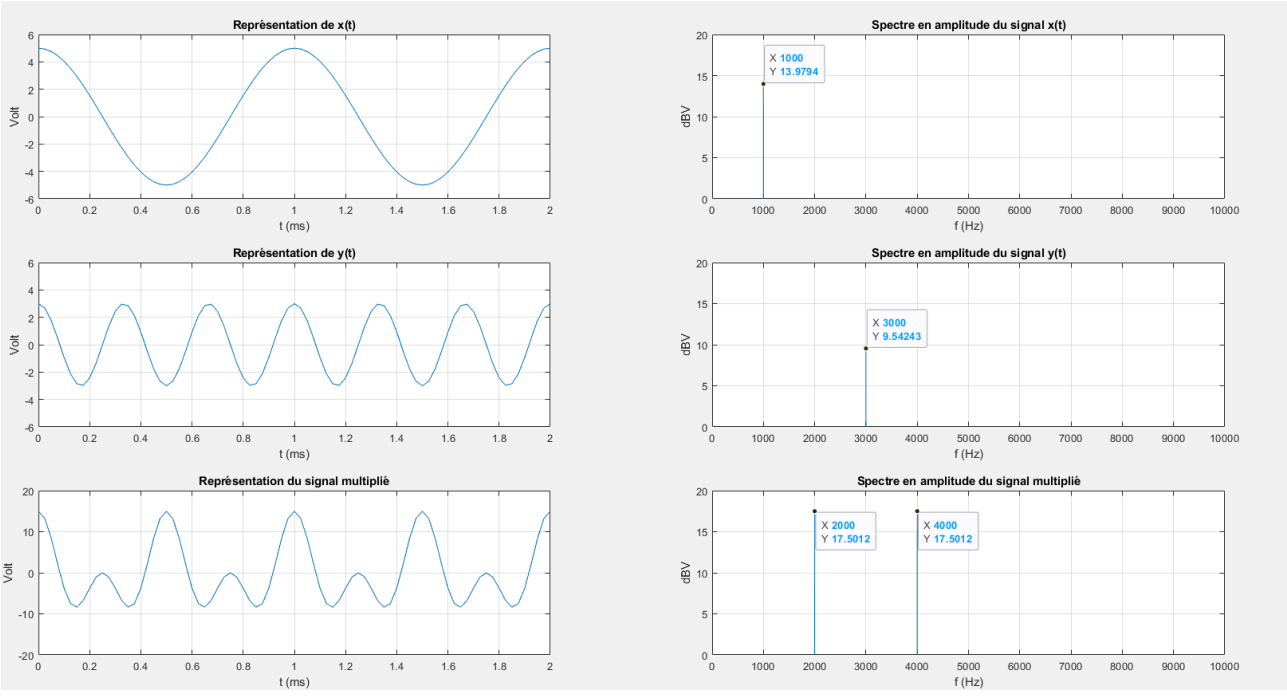
% Signal x2
% Représentation temporelle
subplot(3,2,3);
plot(t * 1000, x2);
title('Représentation de y(t)');
ylabel('Volt'); grid on;
xlabel('t (ms)'); grid on;
xlim([0 2]);
ylim([-6 6]);

% Représentation fréquentielle
subplot(3,2,4);
plot(f, specY);
title('Spectre en amplitude du signal y(t)');
ylabel('dBV'); grid on;
xlabel('f (Hz)'); grid on;
xlim([0 10000]);
ylim([0 20]);

%  Signal multiplié z
% Représentation temporelle
subplot(3,2,5);
plot(t * 1000, z);
title('Représentation du signal multiplié');
ylabel('Volt'); grid on;
xlabel('t (ms)'); grid on;
xlim([0 2]);
ylim([-20 20]);

% Représentation fréquentielle
subplot(3,2,6);
plot(f, specZ);
title('Spectre en amplitude du signal multiplié');
ylabel('dBV'); grid on;
xlabel('f (Hz)'); grid on;
xlim([0 10000]);
ylim([0 20]);
```

Le résultat final est comme ça :



Les résultats obtenus doivent être de la forme suivante :

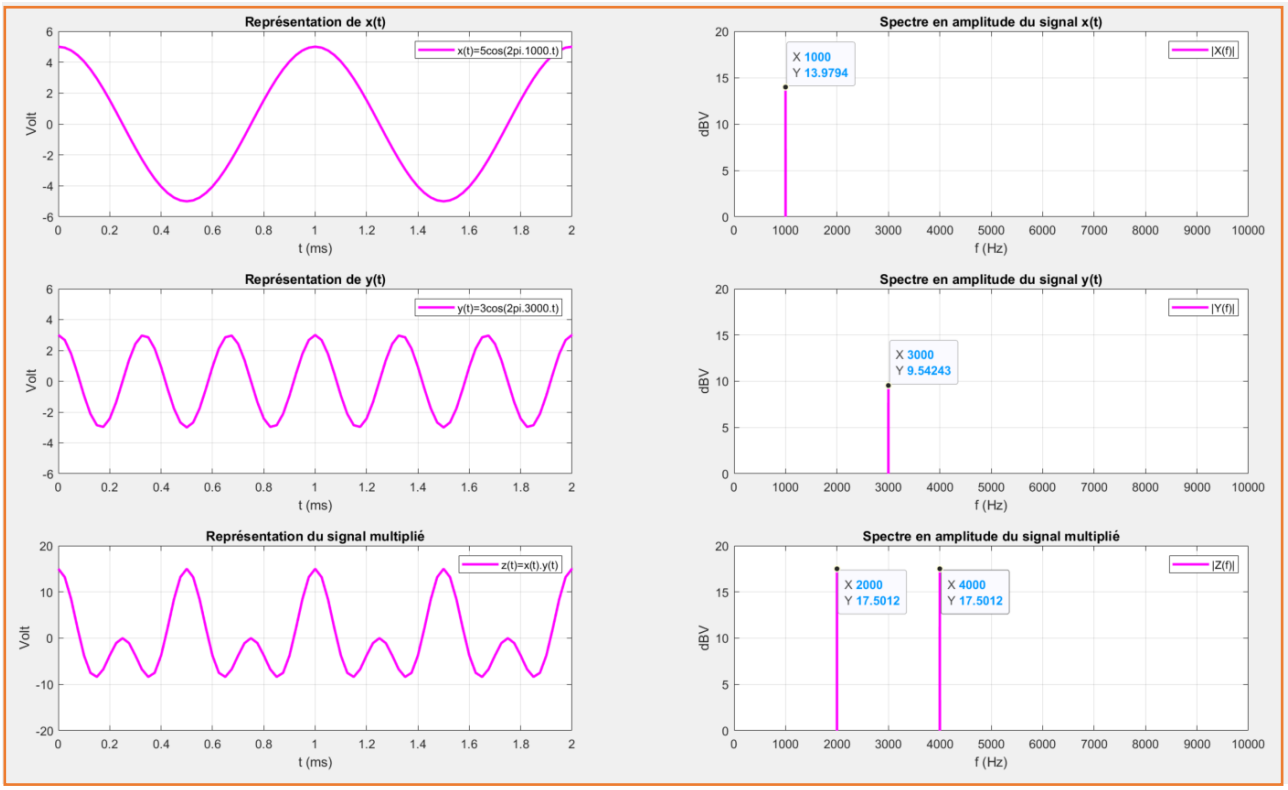


Figure 5: multiplication de deux signaux sinusoïdaux

6) Analyser les résultats obtenus et comparer avec les spectres attendus théoriquement.

Les signaux ont la même forme que les résultats attendus théoriquement il manque juste la couleur et la légende

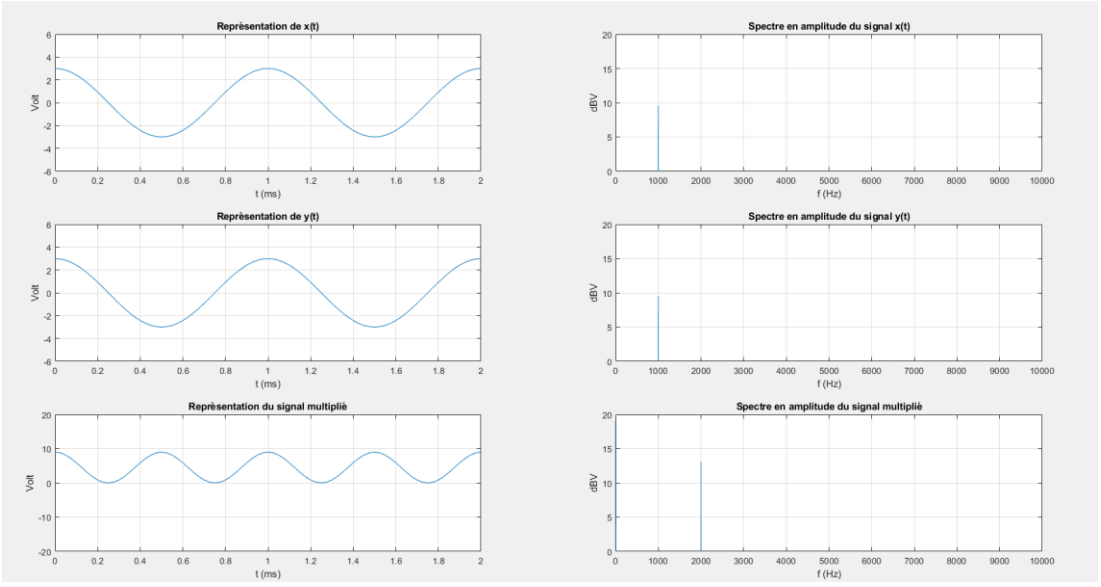
7) Recommencer en prenant maintenant $x1(t) = x2(t) = 3 \times cos(2 \cdot \pi \cdot 1000 \cdot t)$. Analyser. Quelle est la fonction réalisée ?!

Il faut juste remplacer cette partie dans le code ;

A1 = 3; f1 = 1000;

A2 = 3; f2 = 1000;

Quand on exécute le code on a ce résultat



On observe un grand pic a 0 Hz et un autre à 2000 Hz. On obtient un multiplicateur de fréquence.

VII / Somme des éléments d'un tableau

L'objectif est de vous familiariser avec la fonction sum() de Matlab. Cette fonction sera fréquemment utilisée par la suite…

Saisir, commenter puis exécuter le programme suivant :

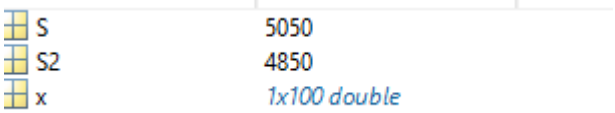
```
clear all;
close all;
clc;

x=(1:100);

S=sum(x);
S2=sum(x(2: end-2));
```

Justifier les valeurs obtenues pour S et S2.

Quand on exécute le programme on a ça :



S = 5050 car la fonction S calcul tous les nombres de x c'est-à-dire qu'il additionne tous les nombres de 1 à 100

S2 = 4850 car elle commence au deuxième élément et finit à l'avant dernier. Elle commence donc à additionner a 2 et finit a 98 et si on fait 5050 – (1+99+100) = 4850

Partie 2 : Calcul numérique d'une intégrale

I / Présentation du contexte

Soit $f(t)$ une fonction continue définie sur un intervalle $t \in [a; b]$. On souhaite évaluer numériquement l'intégrale : $I = \int_a^b f(t) \cdot dt$

Pour donner une valeur approchée, on prendra $\boldsymbol{n}$ **échantillons** sur la courbe uniformément répartis dans l'intervalle $[a;\ b]$. On va donc découper $[a;\ b]$ en $\boldsymbol{n-1}$ **sous-intervalles** de longueur $\boldsymbol{dt = \frac{(b-a)}{n-1}}$ (cf. Figure 1).

Le i[ième] échantillon sera ainsi à l'abscisse : $\boldsymbol{t_i = a + (i-1).dt}$ où i est un nombre entier compris entre 0 et n, ce qui s'écrit : $i \in \langle 1\ ;\ n \rangle$.

Les méthodes de calcul numérique d'intégrales consistent à approximer, sur chaque sous-intervalle, la fonction $f(t)$ par un polynôme.

Dans ce qui suit, on va essayer d'évaluer numériquement l'intégrale : $\boldsymbol{I = \int_0^1 e^{2t} \cdot dt}$

L'objectif est d'évaluer cette intégrale suivant plusieurs méthodes puis de comparer ces méthodes en étudiant leurs vitesses de convergence.

II / Méthode des rectangles à gauche

La méthode des rectangles à gauche consiste à remplacer sur chaque sous-intervalle $J_i = [t_i\ ;\ t_{i+1}]$, la fonction $f(t)$ par $f(t_i)$.

La Figure 1 illustre le calcul de $I = \int_0^1 e^{2t} dt$ par la méthode des rectangles avec **n=5 échantillons**, soit $\boldsymbol{n-1 = 4}$ **intervalles**…

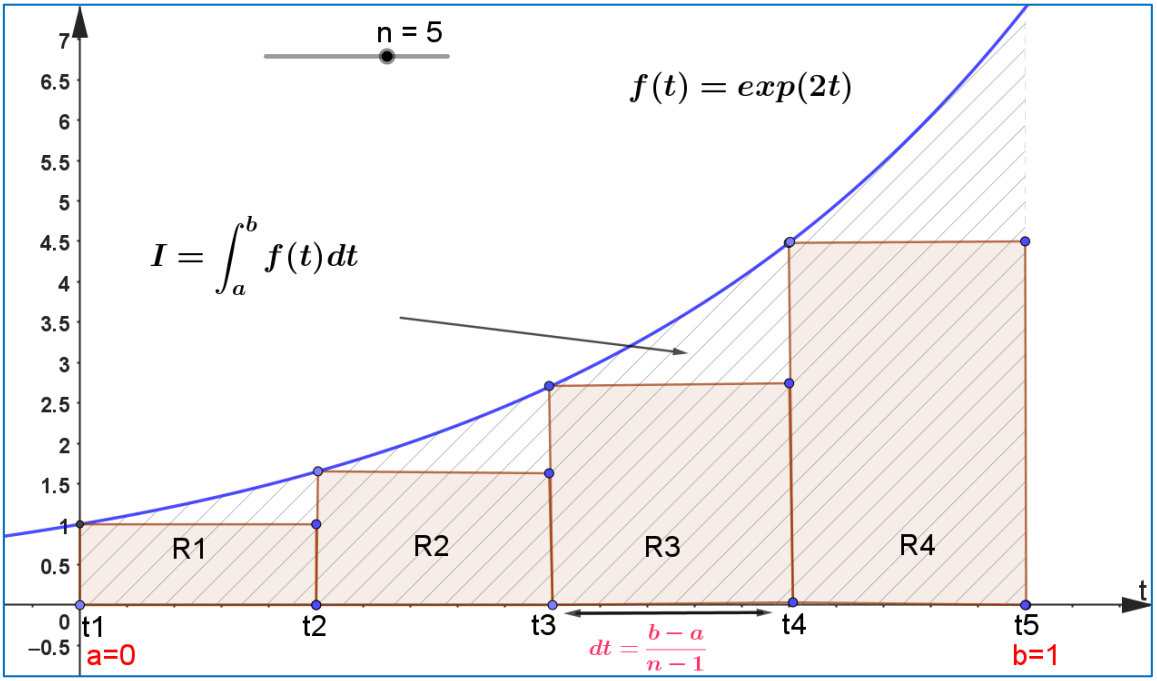


Figure 1: intégration par la méthode des rectangles

Selon cette méthode, la valeur approchée de I avec $n = 5$ points vaut $R = R_1 + R_2 + R_3 + R_4$.

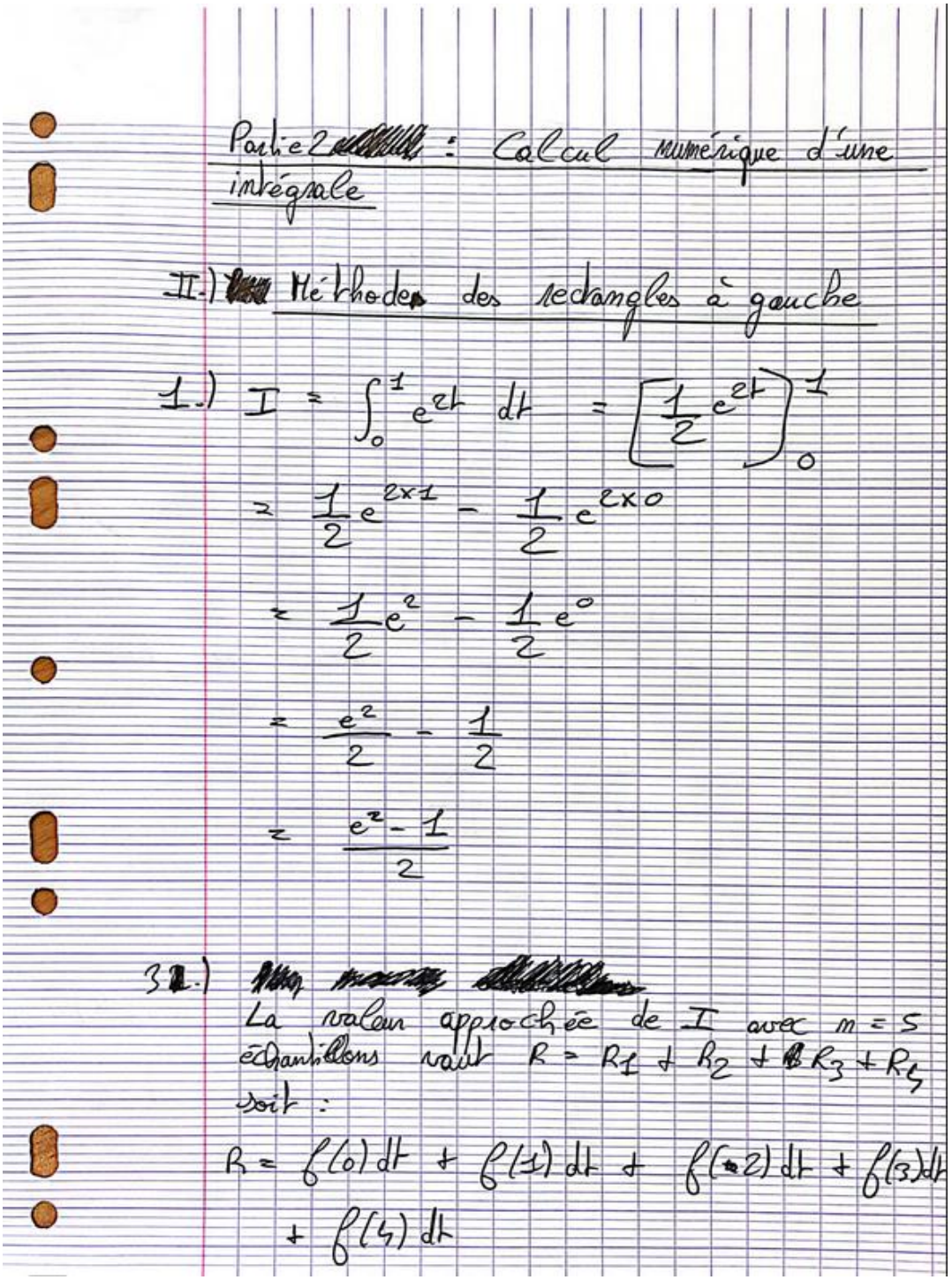
Soit : $R = f(t_1) \times dt + f(t_2) \times dt + f(t_3) \times dt + f(t_4) \times dt$ (eq 1)

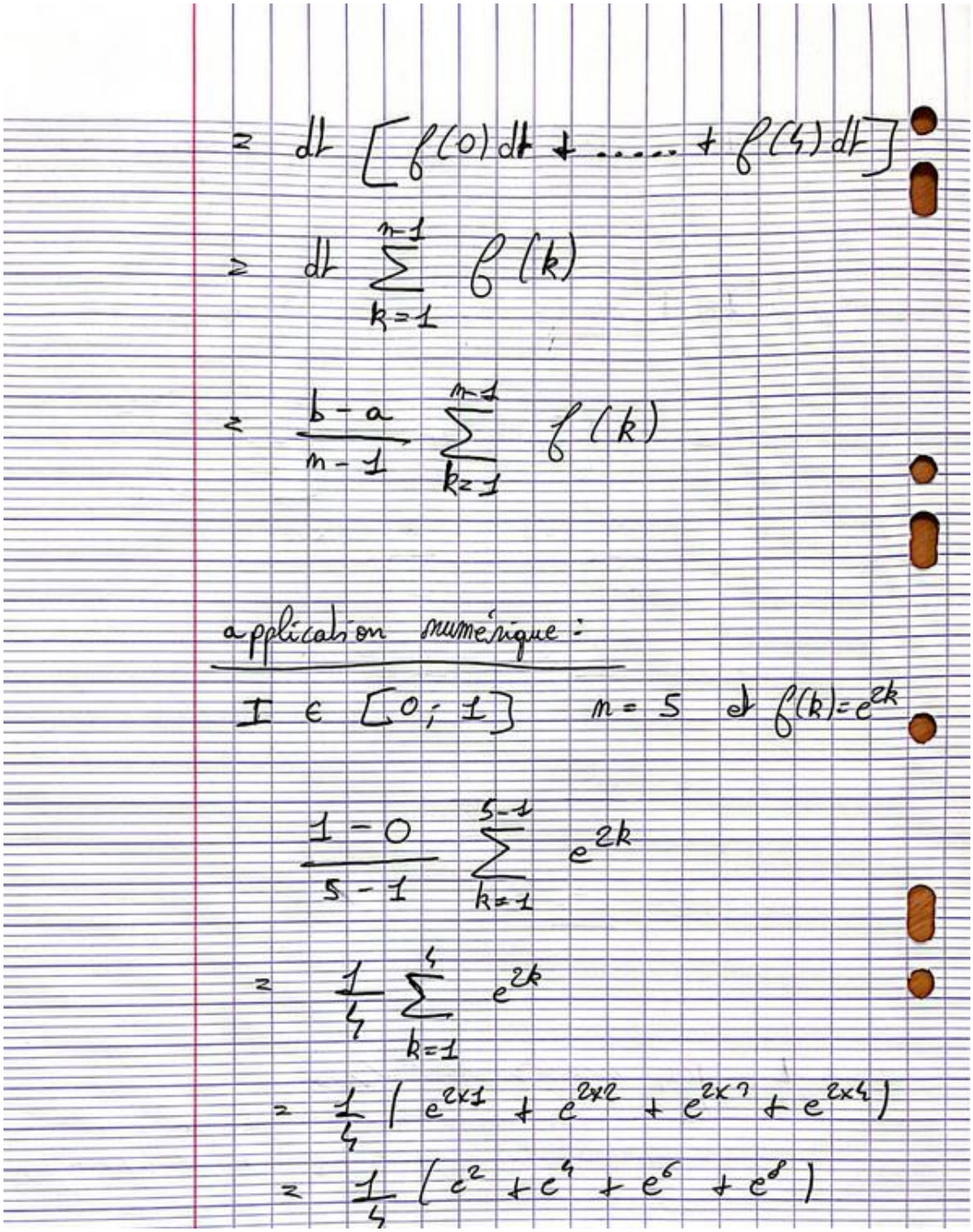
Préparation théorique :

1) Calculer théoriquement la valeur de I. Faire une application numérique à 10^{-5} près.

2) Pour **n=5 échantillons**, calculer R à l'aide d'un tableur (valeur approchée de I selon la méthode des rectangles) (eq 1).

3) Montrer que de manière générale la formule de l'approximation de l'intégrale I par la méthode des rectangles avec n échantillons est :





a	0
b	1
n	5
dt	0,25

k	t	f(t)
0	0	1
1	0,25	1,64872127
2	0,5	2,71828183
3	0,75	4,48168907
4	1	7,3890561
5	1,25	12,182494
6	1,5	20,0855369
7	1,75	33,115452
8	2	54,59815
9	2,25	90,0171313

I	3,19452805
R	2,46217304

$$R = \frac{b-a}{n-1}\sum_{i=1}^{n-1} f(t_i)$$

Réalisation :

On se propose maintenant d'implanter le calcul de cette intégrale en utilisant Matlab.

- **Compléter le script suivant** afin d'implanter l'approximation de la méthode des rectangles sur Matlab.

```
clear all;
close all;
clc;
%% Définition de la fonction et des bornes d'intégration
f = @(x) exp(2*x);  % fonction étudiée
a = 0;
b = 1;

%% Echantillonnage de la fonction sur n points
n = 11;
dt = (b-a)/(n-1); % largeur d'un intervalle.
t = a+(0:n-1)*dt;
y = f(t);
plot(t,y,'-bo','LineWidth',2);

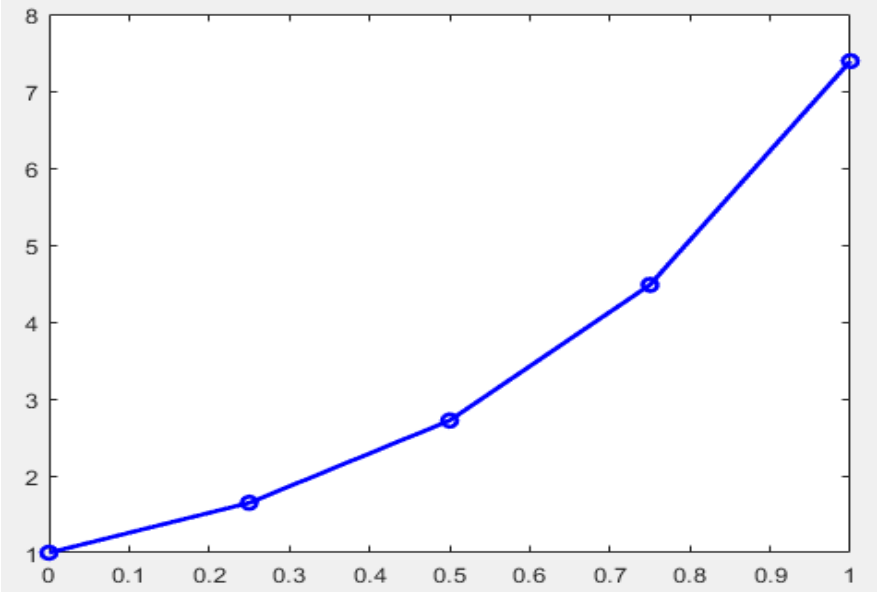
%% méthode des rectangles
% La somme s'arrête a n-1 pour la méthode des rectangles a gauche
R = dt * sum(y(1:n-1));

% Affichage dans la console
disp(['Valeur approchée (n=', num2str(n), ') : R = ', num2str(R)])

%% Validation et calcul de l'erreur relative
% 1. Calcul de la valeur théorique exacte (Primitive de exp(2t) = 1/2 *
exp(2t))
I = (exp(2*1) - exp(2*0)) / 2;
disp(['Valeur theorique exacte : I = ', num2str(I)])

% 2. Calcul de l'erreur relative en %
erreur_relative = abs((I - R) / I) * 100;
disp(['Erreur relative : ', num2str(erreur_relative), ' %'])
```

- Valider le résultat pour $n = 5$.



✋ Calculer l'erreur relative en % dans votre programme. Relever cette erreur pour $n = 1001$.

Quand n = 1001 l'erreur relative est de 0.1 % ce qui est très peu.

III / Méthodes des trapèzes

On se rend compte que la méthode des rectangles peut être améliorée. La méthode des trapèzes consiste à approximer la fonction $f(t)$ dans chacun des sous intervalles par des segments. La figure 2 illustre cette méthode pour $n = 5$ échantillons soit 4 intervalles.

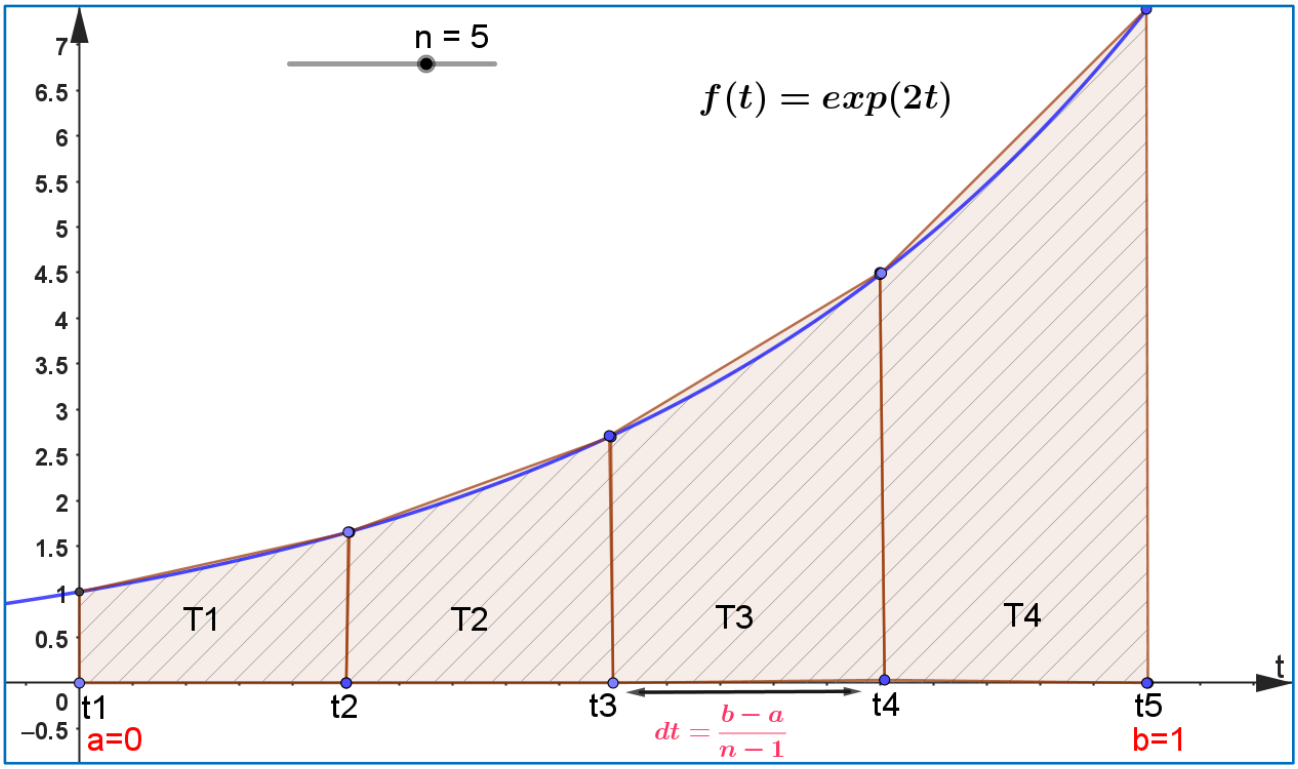
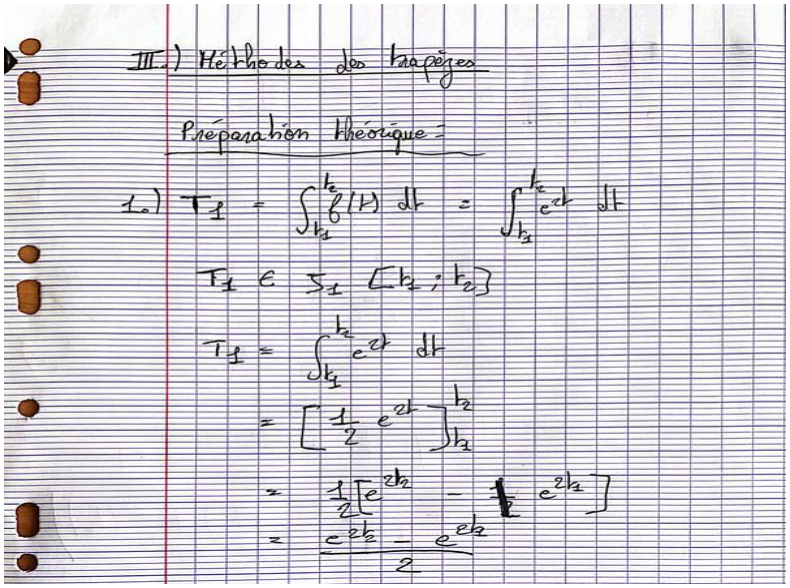


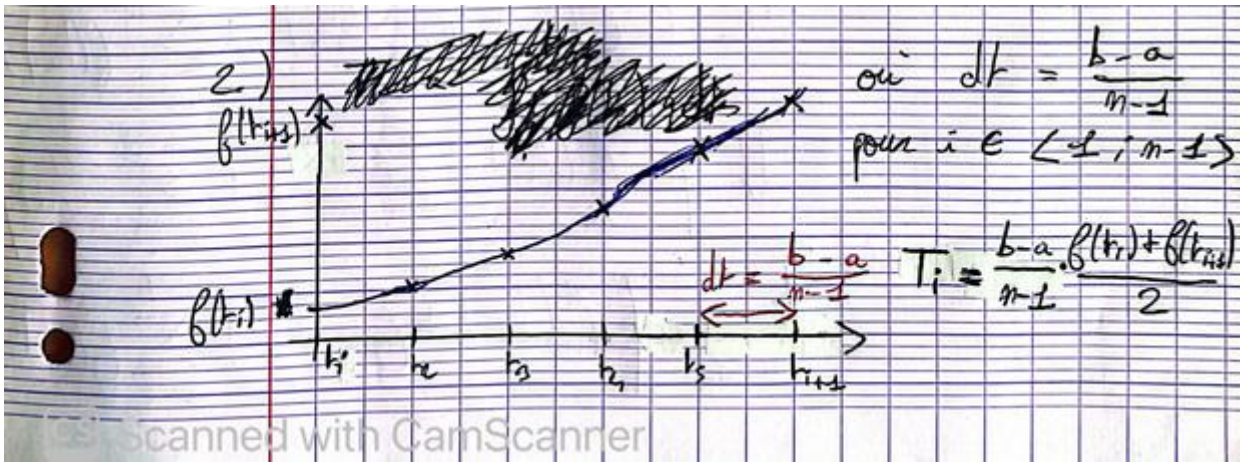
Figure 2: intégration par la méthode des trapèzes

Préparation théorique :

1) Calculer l'aire du trapèze $T1$, sur l'intervalle $J_1 = [t_1 \; ; \; t_2]$.

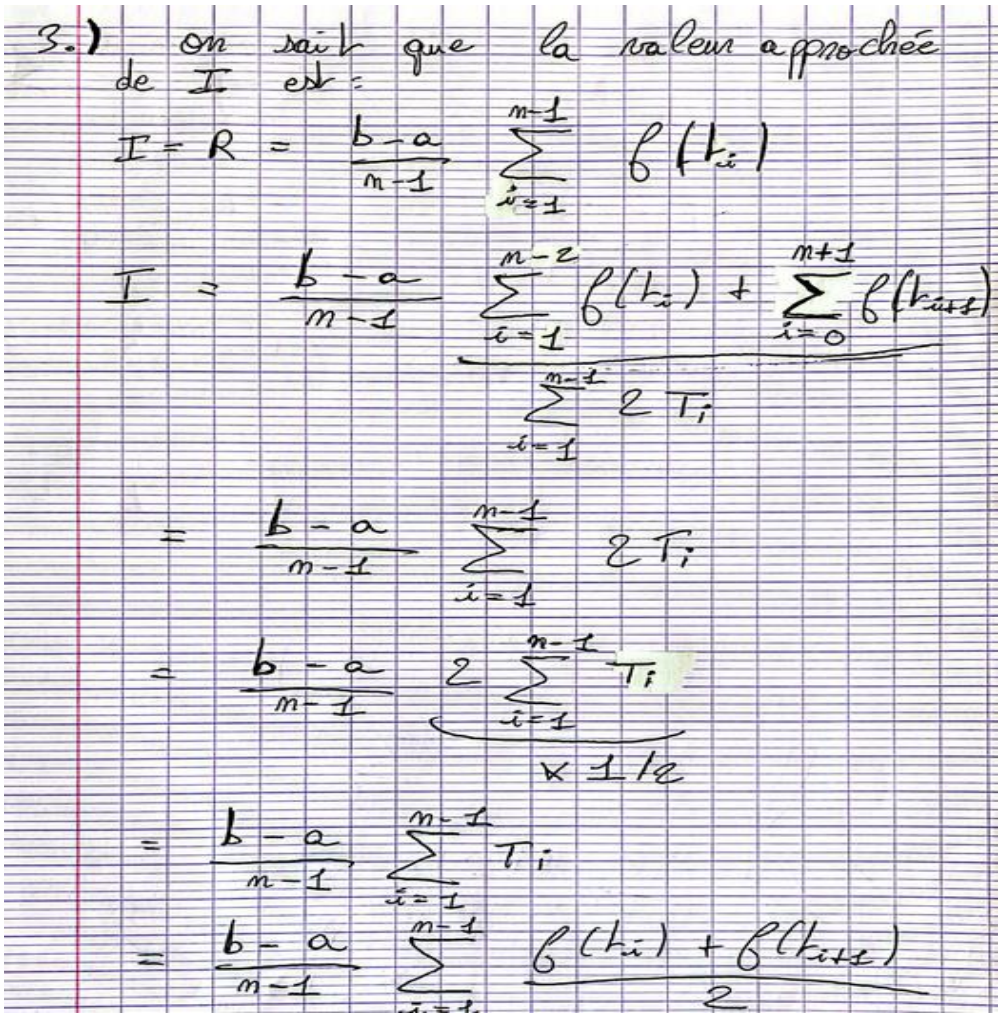


2) De façon plus générale, que vaut l'aire du trapèze Ti dans l'intervalle $J_i = [t_i\ ;\ t_{i+1}]$ pour $i \in \langle 1\ ;\ n-1 \rangle$? Exprimer le résultat en fonction de $dt = \frac{b-a}{n-1}$, $f(t_i)$ et $f(t_{i+1})$



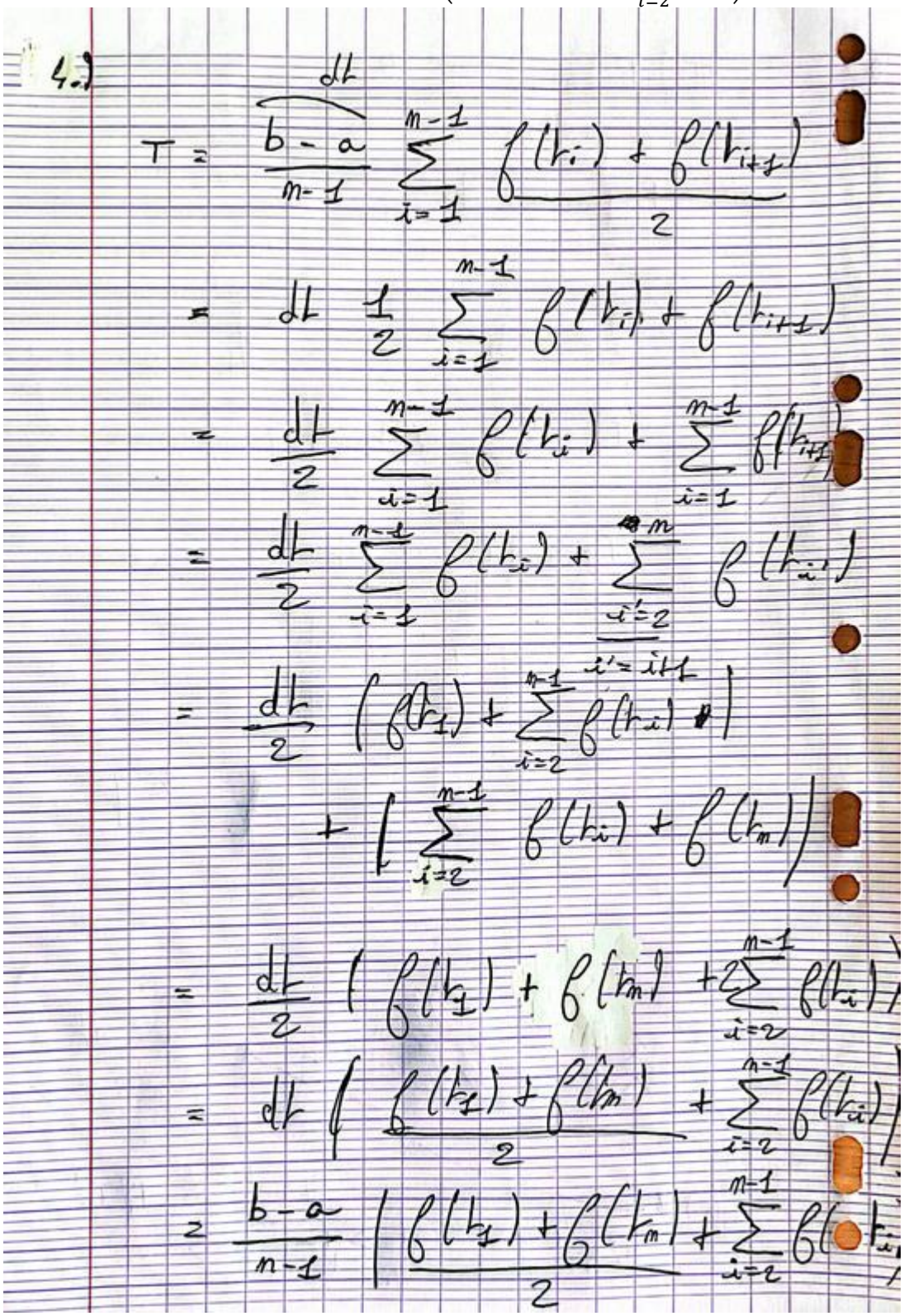
3) Montrer que la formule du calcul approché de l'intégrale I par la méthode des trapèzes est :

$$T = \frac{b-a}{n-1}\sum_{i=1}^{n-1}\frac{f(t_i)+f(t_{i+1})}{2} \qquad (eq\ 2)$$

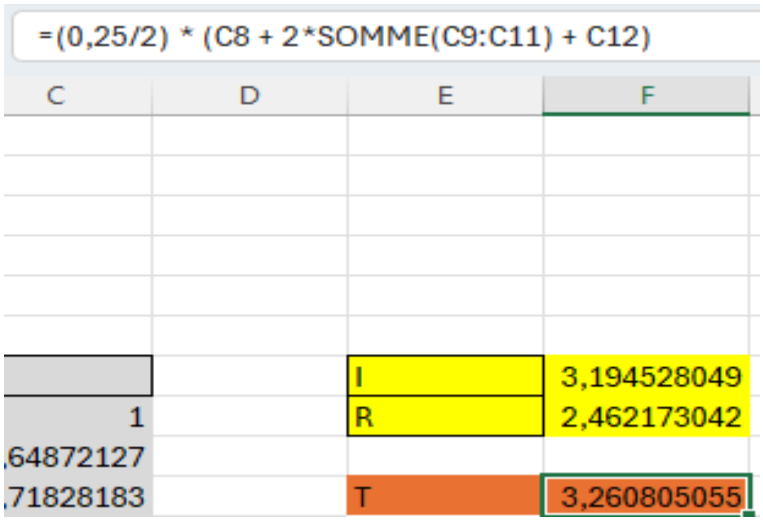


4) (*) Montrer que cette formule peut se mettre sous la forme suivante :

$$T = \frac{b-a}{n-1}\left(\frac{f(t_1)+f(t_n)}{2} + \sum_{i=2}^{n-1} f(t_i)\right)$$



5) Pour $n = 5$, calculer à l'aide d'un tableur la valeur approchée de I, nommée T, par la méthode des trapèzes.



Réalisation :

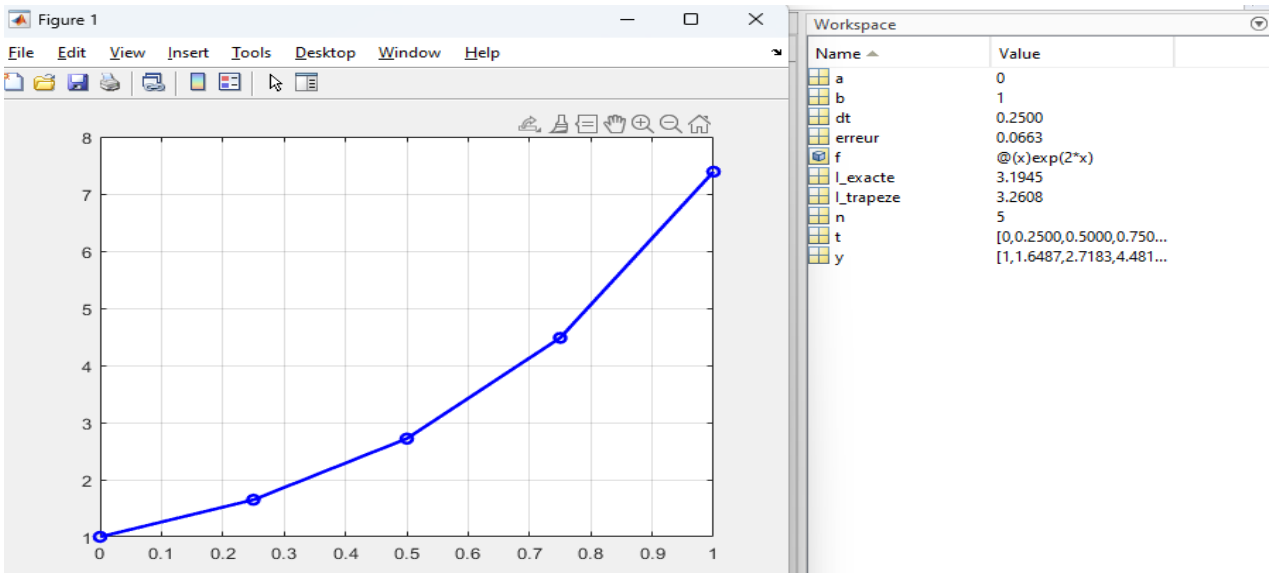
- **Implanter** la méthode des trapèzes sur Matlab.

```
%% Implémentation de la méthode des trapèzes
I_trapeze = (dt / 2) * (y(1) + 2 * sum(y(2:end-1)) + y(end));

%% Affichage des résultats et comparaison avec la valeur exacte
% La primitive de exp(2x) est 0.5 * exp(2x)
I_exacte = 0.5 * (exp(2*b) - exp(2*a));
erreur = abs(I_exacte - I_trapeze);

fprintf('--- Résultats de l''intégration ---\n');
fprintf('Valeur approchée (Trapèzes) : %f\n', I_trapeze);
fprintf('Valeur exacte analytique    : %f\n', I_exacte);
fprintf('Erreur d''approximation       : %e\n', erreur);
```

- Valider le résultat pour $n = 5$.



- Calculer l'erreur relative en % dans votre programme. Relever cette erreur pour $n = 1001$.

```
erreur_relative = abs((I_exacte - I_trapeze) / I_exacte) * 100;

fprintf('Erreur relative             : %.2f %%\n', erreur_relative);
```

L'erreur relative est de 3.3E-5 %

IV / Méthode de Simpson

C'est une des méthodes numériques des plus performantes.

- En faisant des recherches sur Internet (par exemple sur le site https://fr.wikipedia.org/wiki/M%C3%A9thode_de_Simpson) , expliquer le principe de cette méthode.

Géométriquement, au lieu d'utiliser des lignes droites rigides comme la méthode des trapèzes, la méthode de Simpson utilise une courbe "flexible" pour épouser parfaitement les arrondis de votre fonction. Mathématiquement, pour dessiner cette courbure au plus juste, elle a besoin d'étudier trois points à la fois : le début, la fin, et le point central de l'intervalle. Dans le calcul final, la formule accorde quatre fois plus d'importance à ce point du milieu, car c'est lui qui dicte la "rondeur" de la courbe, ce qui rend l'approximation de l'aire incroyablement précise.

- **Facultatif :** implanter cet algorithme. Comparer l'erreur relative pour n=1001 points.

V / Facultatif : Méthode de Monte-Carlo (*)

Principe de la méthode de Monte Carlo :

Cette méthode consiste dans un premier temps à considérer un rectangle (en bleu sur la figure 3) qui inclue la surface de l'intégrale à calculer. La surface S_R du rectangle est facile à calculer.

On génère aléatoirement n points dans ce rectangle.
Parmi ces n points, on détermine le nombre de points inclus dans la zone d'intégration (en rouge). On note $n1$ ce nombre de points.
L'aire de la surface est approximée par : $\boldsymbol{I_{Monte\ Carlo} = \frac{n1}{n}.S_r}$

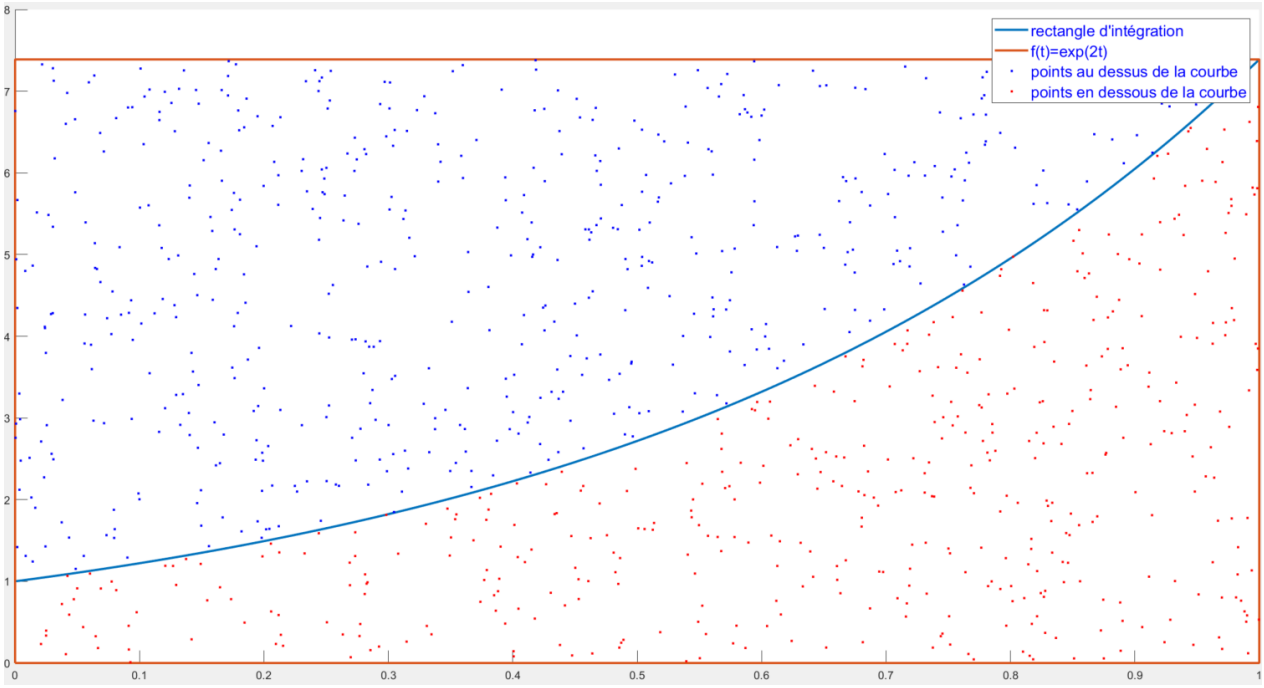


Figure 3: intégration par la méthode de Monte Carlo

Réalisation :

✋ Implanter le calcul de l'intégrale I en utilisant la méthode de Monte-Carlo

```
%% Paramètres de la méthode de Monte-Carlo
n = 1001; % Nombre de points aléatoires demandé dans le sujet

% On définit le rectangle englobant
% La fonction exp(2x) est croissante, son maximum sur [0,1] est donc en x=1
ymax = f(b);
S_R = (b - a) * ymax; % Surface du rectangle d'intégration

%% Génération de n points aléatoires dans le rectangle
% La fonction rand(1, n) génère n nombres aléatoires entre 0 et 1.
x_rand = a + (b - a) * rand(1, n); % Abscisses aléatoires entre a et b
y_rand = ymax * rand(1, n);        % Ordonnées aléatoires entre 0 et ymax

%% Détermination des points sous la courbe
% On vérifie pour chaque point si son ordonnée (y) est sous la courbe (f(x))
points_sous_courbe = (y_rand <= f(x_rand)); % Tableau de VRAI (1) ou FAUX (0)
n1 = sum(points_sous_courbe);               % Nombre de points validés

%% Calcul de l'intégrale approchée
I_MonteCarlo = (n1 / n) * S_R;

%% Calcul de l'erreur relative
I_exacte = 0.5 * (exp(2*b) - exp(2*a));
erreur_relative = abs((I_exacte - I_MonteCarlo) / I_exacte) * 100;

%% Affichage dans la console
fprintf('--- Méthode de Monte-Carlo (n = %d) ---\n', n);
fprintf('Valeur approchée (Monte-Carlo) : %f\n', I_MonteCarlo);
fprintf('Valeur exacte analytique       : %f\n', I_exacte);
fprintf('Erreur relative                : %.2f %%\n', erreur_relative);

%% Tracé graphique (Pour reproduire la Figure 3 de votre document)
figure;
hold on; grid on;

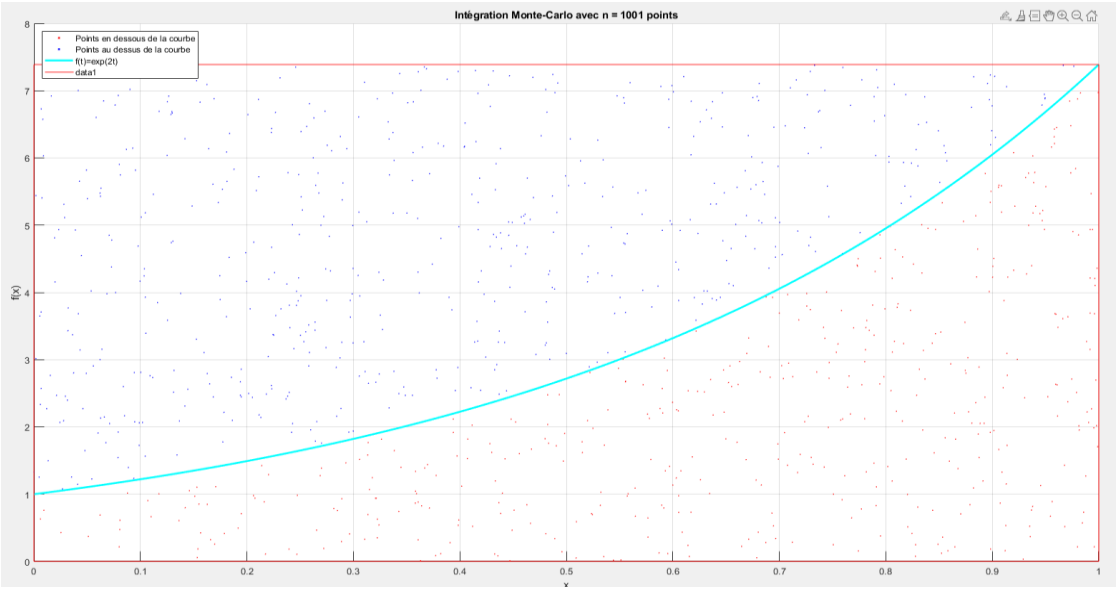
% Tracer les points aléatoires (Rouge = sous la courbe, Bleu = au-dessus)
% Le symbole ~ sert à prendre l'inverse (les points qui NE SONT PAS sous la courbe)
scatter(x_rand(points_sous_courbe), y_rand(points_sous_courbe), 10, 'r', '.');
scatter(x_rand(~points_sous_courbe), y_rand(~points_sous_courbe), 10, 'b', '.');

% Tracer la courbe de la fonction par-dessus
t = linspace(a, b, 100); % Création d'un vecteur lisse pour tracer la courbe
plot(t, f(t), '-c', 'LineWidth', 2);

% Mise en forme du graphique
title(sprintf('Intégration Monte-Carlo avec n = %d points', n));
xlabel('x'); ylabel('f(x)');
legend('Points en dessous de la courbe', 'Points au dessus de la courbe', 'f(t)=exp(2t)', 'Location', 'northwest');
% Tracer la limite du rectangle d'intégration
plot([a b b a a], [0 0 ymax ymax 0], 'r-');
```

Remarque : cette méthode est particulièrement adaptée au calcul d'intégrales multidimensionnelles (volumes…). En 2D, elle a une vitesse de convergence très faible.

On a ce resultat :



✋ Calculer l'erreur relative en % dans votre programme. Relever cette erreur pour $n = 1001$.

L'erreur relative est de 4.67%

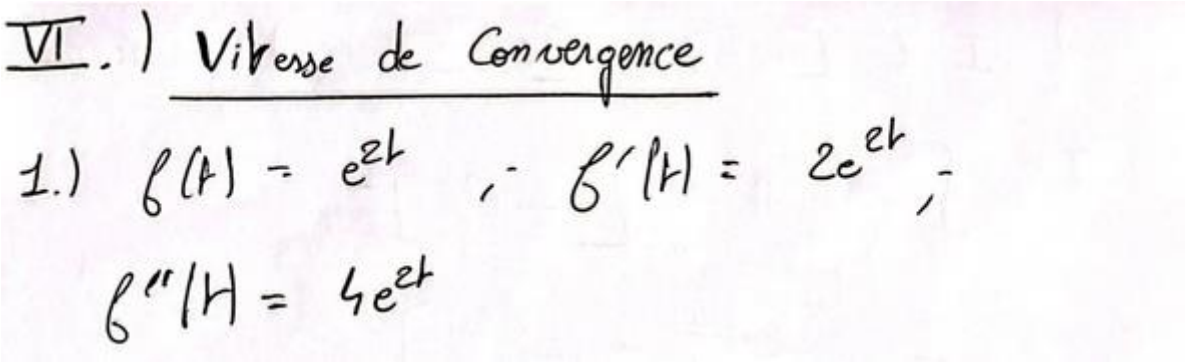
VI / Vitesse de convergence (*)

Dans cette partie, on prendra toujours $I = \int_0^1 e^{2t} dt$.

On s'intéresse aux erreurs d'approximations de l'Intégrale I en fonction du nombre de points, n.

On note : $m_1 = \sup_{[a,b]} |f'(t)|$, la valeur maximale de f' sur l'intervalle $[a, b] = [0,1]$.
De même on note, $m_2 = sup_{[a,b]} |f''(t)|$

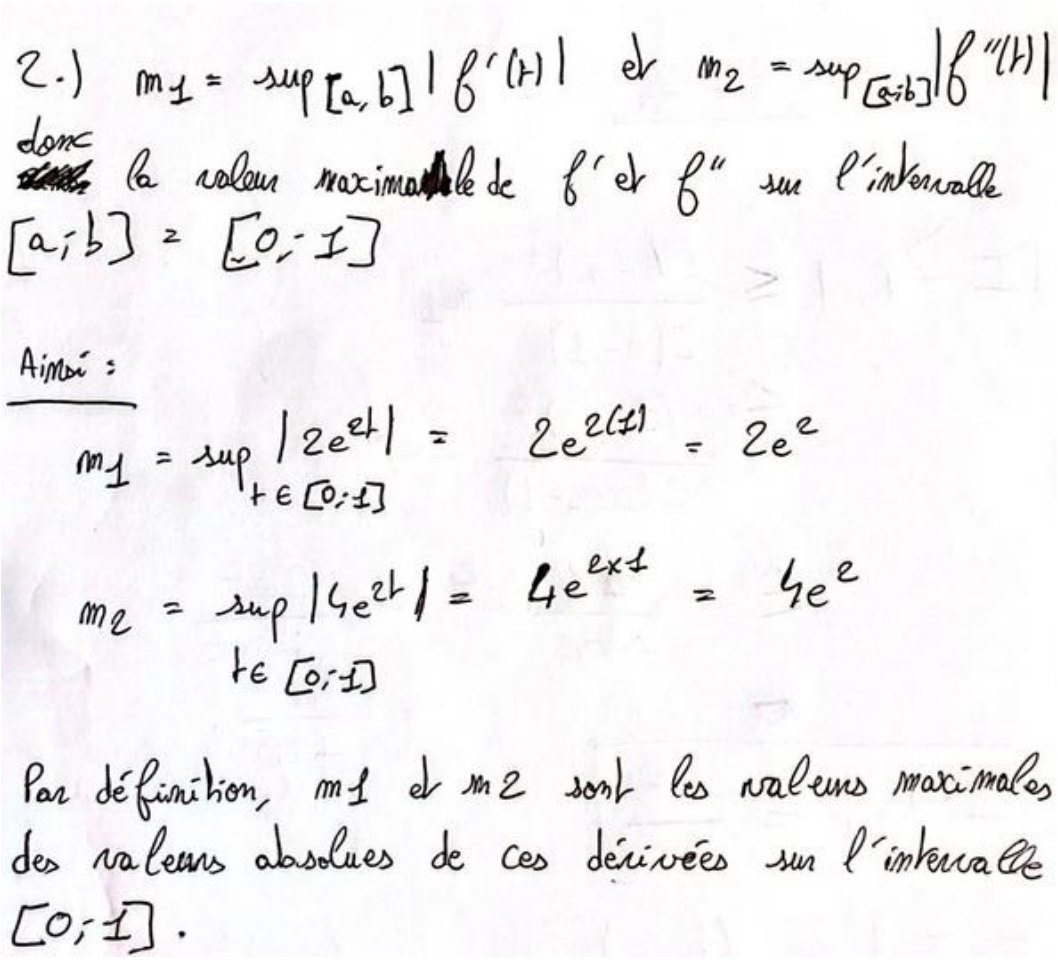
1) Déterminer la formule de $f'(t)$ et $f''(t)$.


VI.) Vitesse de Convergence

1.) $f(t) = e^{2t}$; $f'(t) = 2e^{2t}$;

$f''(t) = 4e^{2t}$

2) Montrer que : $m_1 = 2e^2$ et $m_2 = 4e^2$


2.) $m_1 = \sup_{[a,b]} |f'(t)|$ et $m_2 = \sup_{[a;b]} |f''(t)|$

donc la valeur maximale de f' et f'' sur l'intervalle $[a;b] = [0;1]$

Ainsi :

$m_1 = \sup_{t \in [0;1]} |2e^{2t}| = 2e^{2(1)} = 2e^2$

$m_2 = \sup_{t \in [0;1]} |4e^{2t}| = 4e^{2\times 1} = 4e^2$

Par définition, m1 et m2 sont les valeurs maximales des valeurs absolues de ces dérivées sur l'intervalle $[0;1]$.

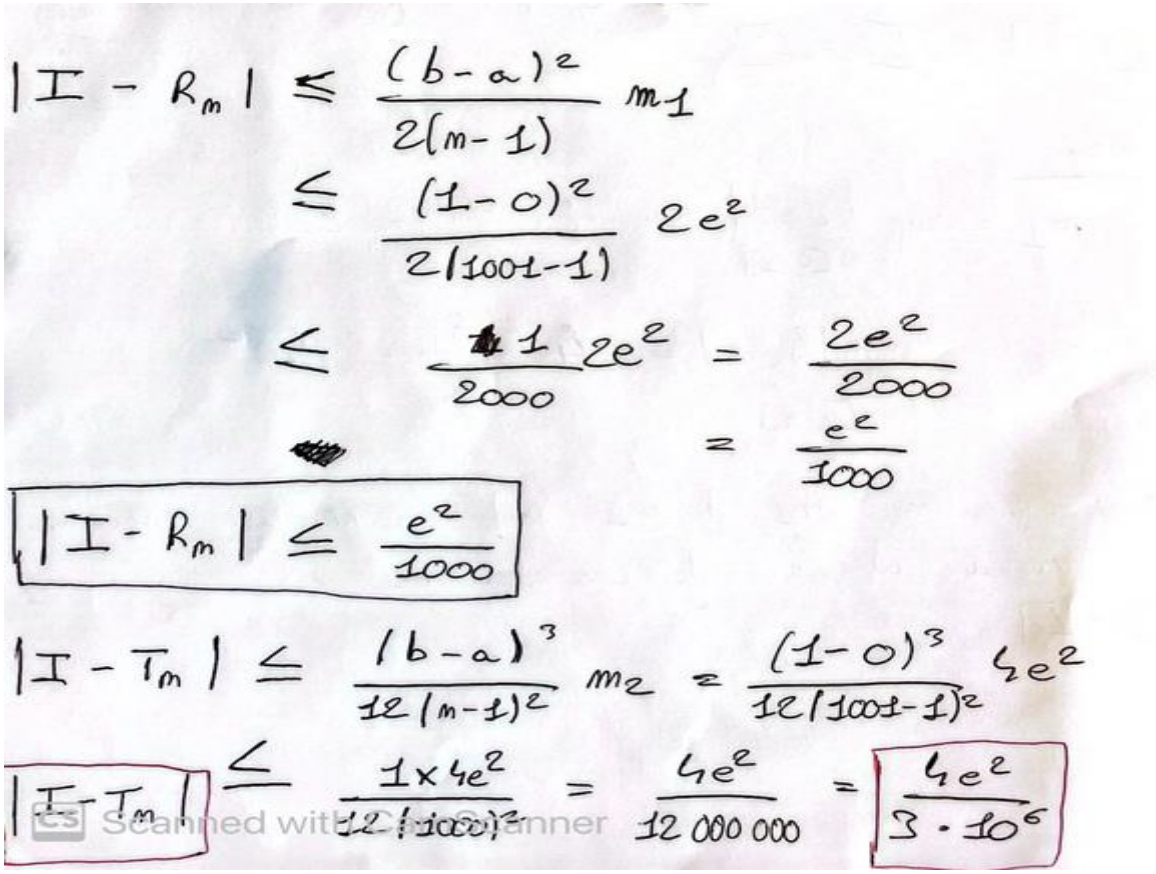
Soit ***Rn* et *Tn*** les approximations sur ***n points*** de l'intégrale I respectivement par la méthode des rectangles et celle des trapèzes.

On donne les formules de majorations suivantes :

$$|I - Rn| \leq \frac{(b-a)^2}{2(n-1)} m_1 \quad (1)$$

$$|I - Tn| \leq \frac{(b-a)^3}{12(n-1)^2} m_2 \quad (2)$$

3) Pour n=1001, justifier que les résultats obtenus dans les questions précédentes par simulation vérifient les majorations (1) et (2).



4) Pour la méthode des rectangles, faire un programme permettant de représenter l'évolution de l'erreur $|I - R|$ en fonction de n pour n variant de 3 à 1000 (par pas de 2). On pourra prendre une échelle logarithmique.

```
%% Initialisation des vecteurs pour n et l'erreur
n_valeurs = 3:2:1000; % n variant de 3 à 1000 par pas de 2
erreurs = zeros(size(n_valeurs)); % Pré-allocation pour optimiser la mémoire

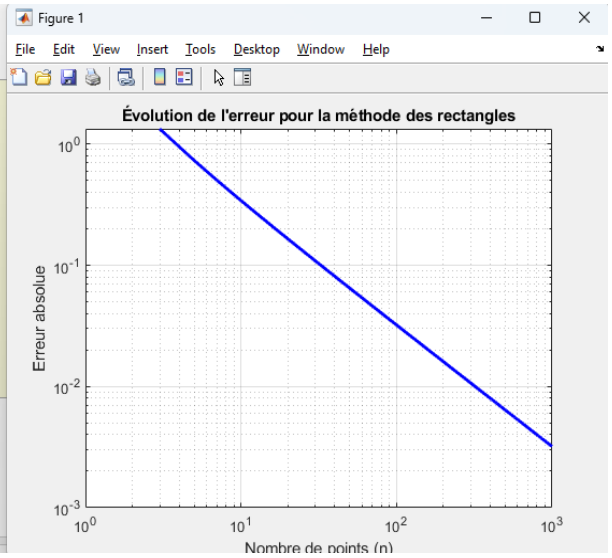
%% Calcul de l'erreur pour chaque n
for k = 1:length(n_valeurs)
    n = n_valeurs(k);
    dt = (b-a)/(n-1); % Largeur d'un intervalle

    % Création du vecteur temps (points à gauche pour les rectangles)
    % On s'arrête à n-2 car il y a n-1 intervalles
    t = a + (0:n-2) * dt;

    % Approximation par la méthode des rectangles (à gauche)
    I_approx = dt * sum(f(t));

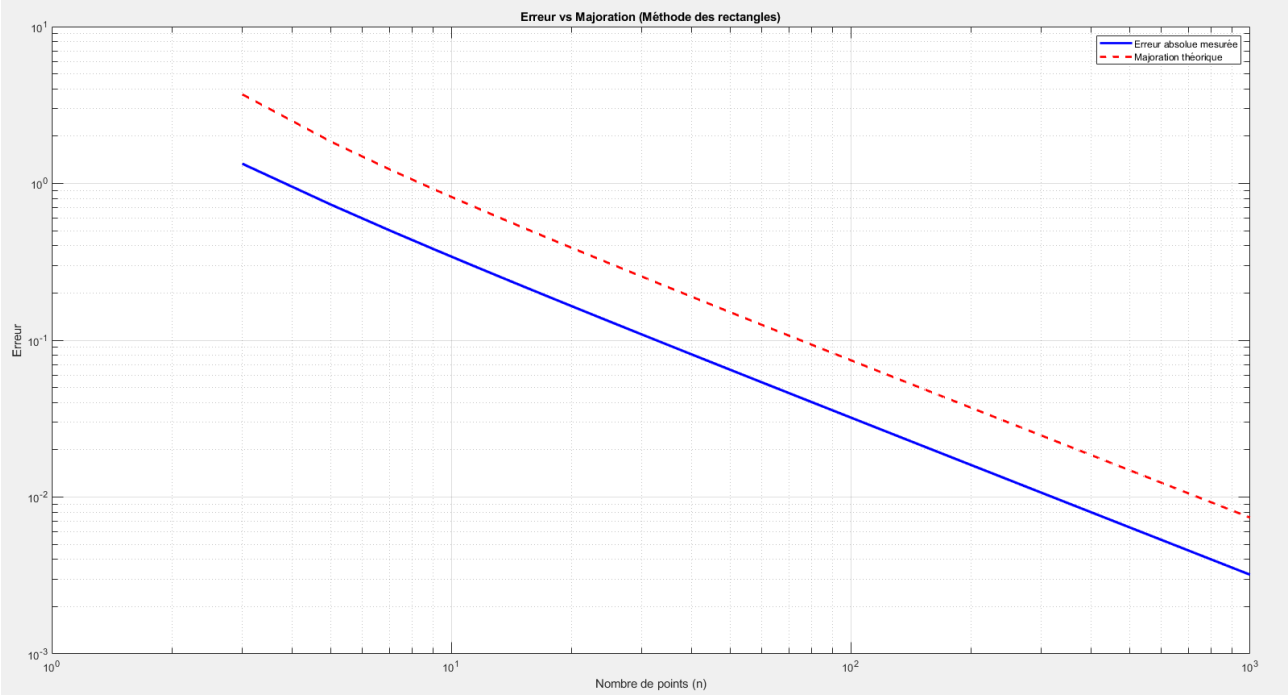
    % Calcul de l'erreur absolue
    erreurs(k) = abs(I_exact - I_approx);
end

%% Représentation graphique avec échelle logarithmique
figure;
loglog(n_valeurs, erreurs, '-b', 'LineWidth', 2);
grid on;
xlabel('Nombre de points (n)');
ylabel('Erreur absolue');
title('Évolution de l''erreur pour la méthode des rectangles');
```



5) Représenter aussi l'évolution en fonction de n de la majoration $\frac{(b-a)^2}{2(n-1)} m_1$. Conclure

```
% Calcul de m1 (maximum de |f'(x)| sur l'intervalle) avant la boucle
m1 = 2 * exp(2*b);
% Calcul de la majoration à insérer dans la boucle for
majoration(k) = ((b-a)^2 / (2*(n-1))) * m1;
```



La courbe de l'erreur reste toujours reste toujours strictement en dessous de la courbe de majoration théorique

6) Faire la même étude pour la méthode des trapèzes.

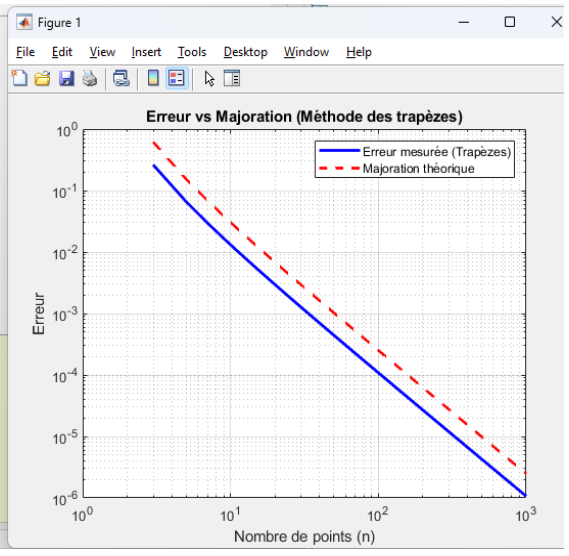
```
%% Calcul de l'erreur et de la majoration pour chaque n
for k = 1:length(n_valeurs)
    n = n_valeurs(k);
    dt = (b-a)/(n-1); % Largeur d'un intervalle
    t = a:dt:b;       % Vecteur temps complet de a à b (n points)

    % Approximation par la méthode des trapèzes
    % Formule vectorisée : on somme tous les points, mais on divise par 2 les extrémités
    I_approx = dt * (sum(f(t)) - (f(a) + f(b))/2);

    % Calcul de l'erreur absolue
    erreurs(k) = abs(I_exact - I_approx);

    % Calcul de la majoration théorique (ordre 2)
    majoration(k) = ((b-a)^3 / (12*(n-1)^2)) * m2;
end

%% Représentation graphique avec échelle logarithmique
figure;
loglog(n_valeurs, erreurs, '-b', 'LineWidth', 2);
hold on;
loglog(n_valeurs, majoration, '--r', 'LineWidth', 2);
grid on;
xlabel('Nombre de points (n)');
ylabel('Erreur');
legend('Erreur mesurée (Trapèzes)', 'Majoration théorique');
title('Erreur vs Majoration (Méthode des trapèzes)');
```



L'erreur est moins importante mais al courbe de majoration reste toujours strictement supérieur a la courbe de l'erreur

7) **Facultatif :** recommencer avec la méthode de Simpson. La formule théorique est donnée par :

$|I - S| \leq \frac{(b-a)^5}{180(n-1)^4} m_4 \quad avec \ m_4 = sup_{[a,b]} \left| f^{(4)}(t) \right|$

8) **Facultatif :** faire aussi l'étude pour la méthode de Monte Carlo.

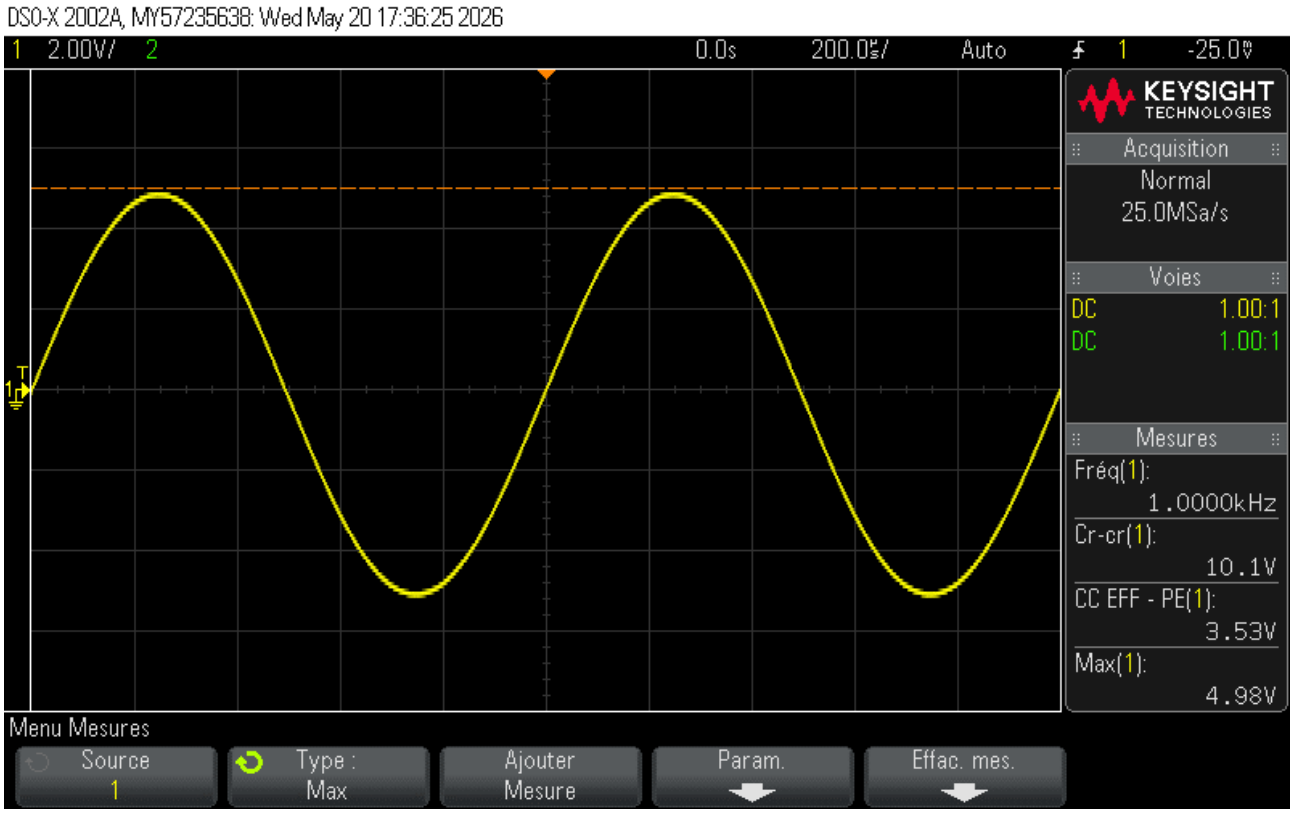
Partie 3 : Etude d'un signal temporel

Objectif : on se propose d'analyser sous Matlab un signal acquis à l'aide d'un oscilloscope. On souhaite le caractériser en effectuant les calculs suivants :

- Mesure de l'amplitude de sa tension.
- Calcul de la valeur moyenne et de la valeur efficace de sa tension.
- Détermination du spectre en amplitude.
- Détermination du fondamental et des harmoniques.
- Calcul de la puissance moyenne en utilisant Parseval.

I / Acquisition des signaux

- Générer à l'aide d'un GBF un signal $x(t)$, sinusoïdal de fréquence 1000Hz et d'amplitude 5V.
- Observer ce signal à l'oscilloscope et le capturer à l'aide du logiciel « Keysight Benchvue ».



- Exporter ce signal en fichier Matlab. Pour cela, aller sur « ***Données de trace*** » puis « ***Exporter*** » (cf. Figure 1). On nommera le fichier exporté « **sinus.mat** ».

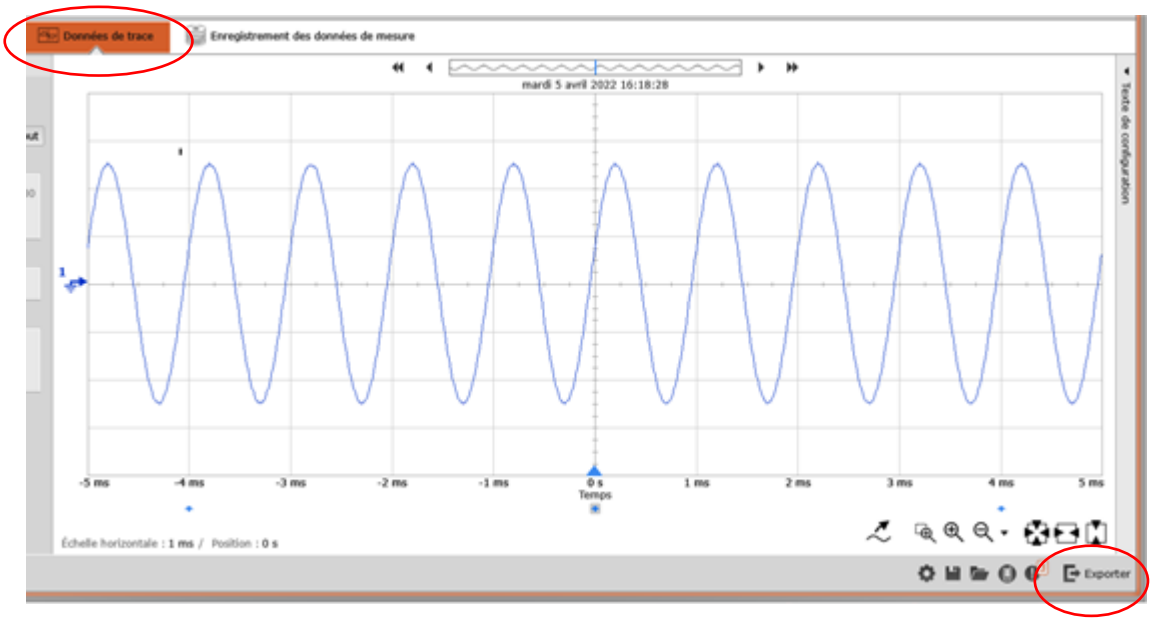
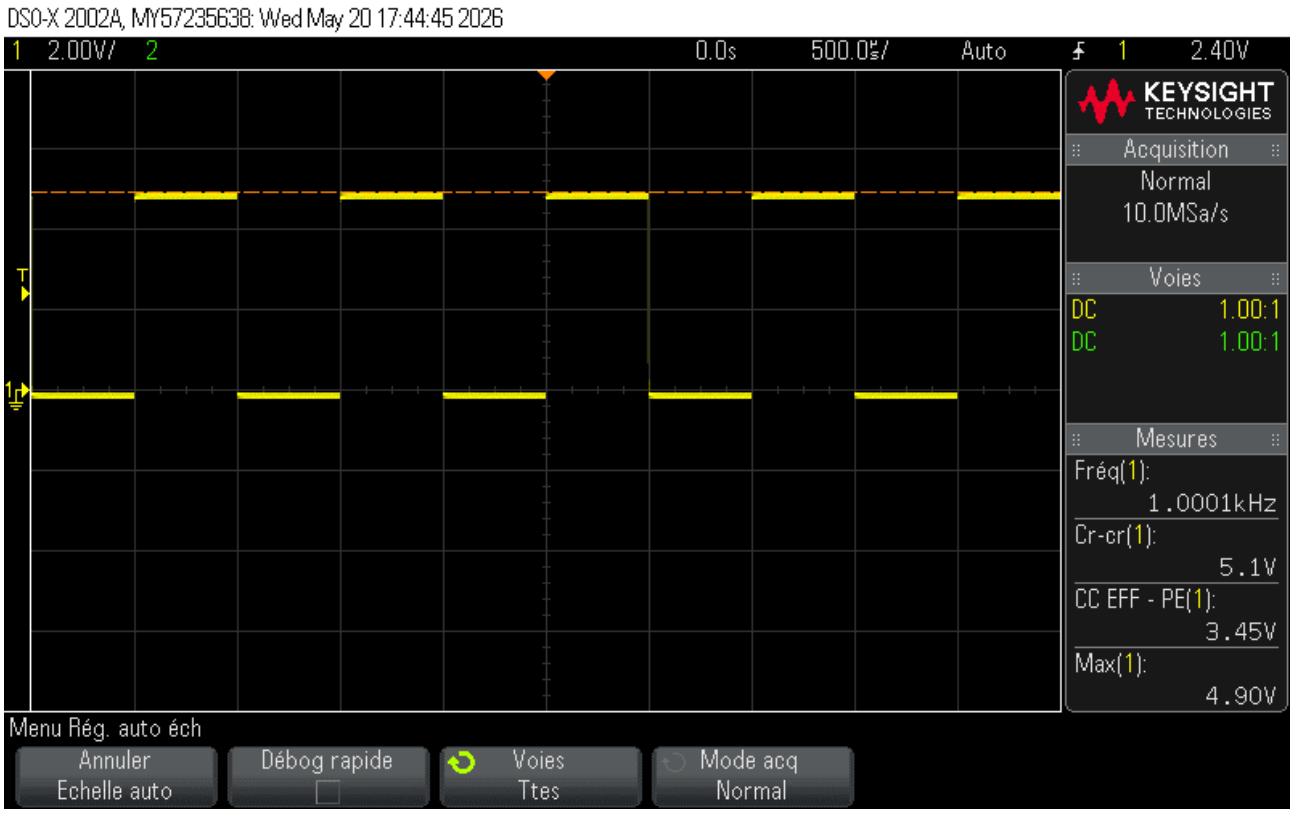


Figure 1 : export du signal de l'oscilloscope

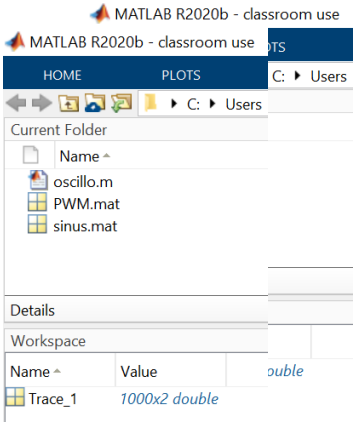
- Recommencer en générant maintenant y(t) : signal carré 0-5V de fréquence 100kHz et de rapport cyclique $R_c = 0{,}2$. Le fichier importé sera nommé « **PWM.mat** ».



II / Etude du signal sinusoïdal : $x(t)$

II.1 / Extraction et affichage

- Créer un nouveau script Matlab, nommé « oscillo.m » et l'enregistrer dans un répertoire identifié.
- Dans ce répertoire ajouter les fichiers « sinus.mat » et « PWM.mat » importés précédemment.
- Récupérer les données avec la procédure suivante :



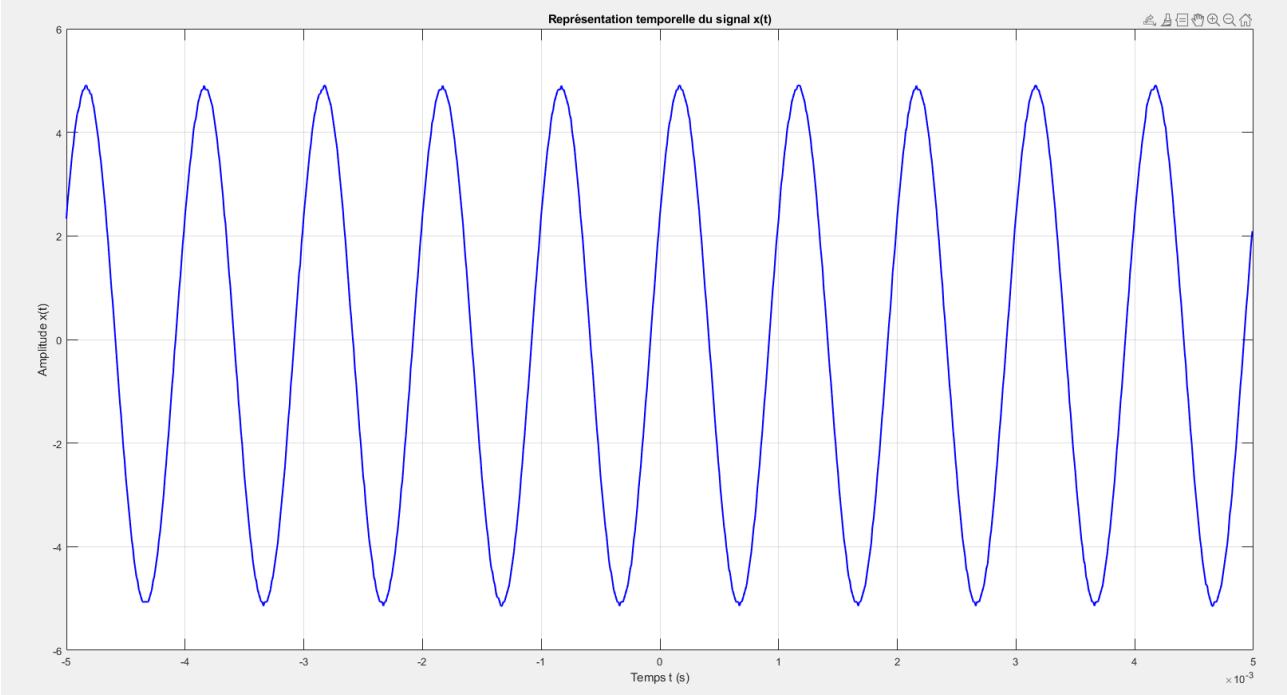

```
close;
clear;
clc;

load sinus.mat
% load PWM.mat
temp=transpose(Trace_1);
t=temp(1,:);
x=temp(2,:);
```

1) Quel est le rôle de la commande « transpose » ?

La commande transpose sert à inverser les lignes et les colonnes d'une matrice (la transposée)s

- Afficher le signal $x(t)$. (Commande plot).



- Récupérer le nombre d'échantillons, Ns, de ce signal à l'aide de la commande « size ».

```
Ns = size(x, 2);
```

On a 1000 échantillons.

✋ A partir du tableau des temps, calculer la période d'échantillonnage : Te.

```
Te = t(2) - t(1);
```

La période d'échantillonnage est e 1 ms

Remarque : Ns et Te seront utiles pour la suite…

II.2 / Exploitation des données du signal x(t)

II.2.a / Calcul de l'amplitude

✋ En utilisant les fonctions « max » et « min » calculer l'amplitude de ce signal.

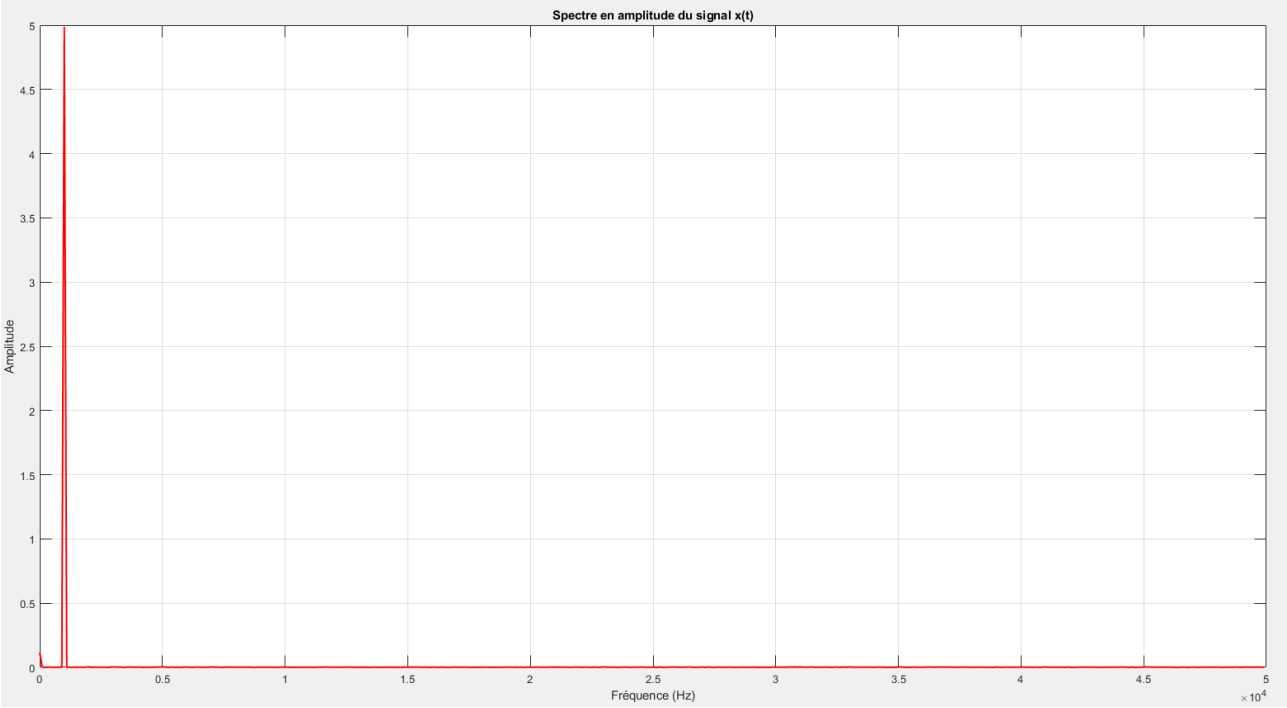
```
%% Recherche des valeurs extrêmes du signal
val_max = max(x);  % Trouve la valeur la plus élevée
val_min = min(x);  % Trouve la valeur la plus basse

%% 1 - Calcul de l'amplitude crête-à-crête
Amp_cc = val_max - val_min;
```

Amp_cc	10.0502
Ns	1000
t	*1x1000 double*
Te	1.0000e-05
temp	*2x1000 double*
Trace_1	*1000x2 double*
val_max	4.9045
val_min	-5.1457
x	*1x1000 double*

II.2.b / Représentation du spectre en amplitude et détermination du fondamental

✋ En utilisant la fonction « spectre » vue précédemment, calculer puis afficher le spectre en amplitude de ce signal x(t).



- A l'aide de la fonction « max » déterminer l'amplitude du fondamental de x(t).

L'amplitude de la fondamentale est de 5V

- En déduire une détermination de la valeur de la fréquence du fondamental : F_0.

Name	Value
A	*1x500 double*
Amp_cc	10.0502
Amp_F0	4.9879
f	*1x500 double*
F0	1.0000e+03
Fe	1.0000e+05
indice_max	11
Ns	1000
t	*1x1000 double*
Te	1.0000e-05
temp	*2x1000 double*
Trace_1	*1000x2 double*
val_max	4.9045
val_min	-5.1457
x	*1x1000 double*

La fréquence de la fondamentale est de 1 kHz

II.2.c / Calcul de la valeur moyenne

La valeur moyenne du signal est donnée par la formule : $x_{moy} = \frac{1}{T} \times \int_0^T x(t) \cdot dt$ où T représente la période de la *composante fondamentale*.

- A partir de la fréquence du fondamental, déterminer la période du signal : $T = \frac{1}{F_0}$

T = 1/1000 = 0.001s

- En déduire le nombre d'échantillons pour une période de x(t) à l'aide de la fonction « round » : $Nsparperiode = M = round\left(\frac{T}{Te}\right)$

M $= round\left(\frac{0.001}{1e-5}\right) = 100$

- Réaliser un programme Matlab qui calcule la valeur moyenne du signal en utilisant la méthode des rectangles.

```
duree_totale = t(end) - t(1);

% Valeur moyenne = Intégrale / Durée
valeur_moyenne = integrale_rectangles / duree_totale;

fprintf('\n--- RESULTATS TEMPORELS ---\n');
fprintf('Valeur moyenne (méthode des rectangles) : %f\n', valeur_moyenne);
```

valeur_moyenne	-0.1182

II.2.d / Calcul de la valeur efficace

La valeur efficace du signal est donnée par la formule : $x_{eff} = \sqrt{\frac{1}{T} \times \int_0^T x^2(t) \cdot dt}$

- Réaliser un programme Matlab qui calcule la valeur efficace du signal en utilisant la méthode des rectangles.

```
%% 3. Calcul de la valeur efficace par la méthode des rectangles
% 1. On élève chaque point du signal au carré (attention à bien utiliser .^ pour une opération élément par élément)
x_carre = x.^2;

% 2. Calcul de l'intégrale du signal au carré (méthode des rectangles à gauche)
integrale_carre = sum(x_carre(1:end-1)) * Te;

% 3. Application de la formule : racine carrée de (Intégrale / Durée totale)
valeur_efficace = sqrt(integrale_carre / duree_totale);

fprintf('Valeur efficace (méthode des rectangles) : %f\n', valeur_efficace);
```

valeur_efficace 3.5302

III / Etude du signal PWM : y(t)

III.1.a / Etude théorique du signal PWM

On considère le signal y(t) suivant :

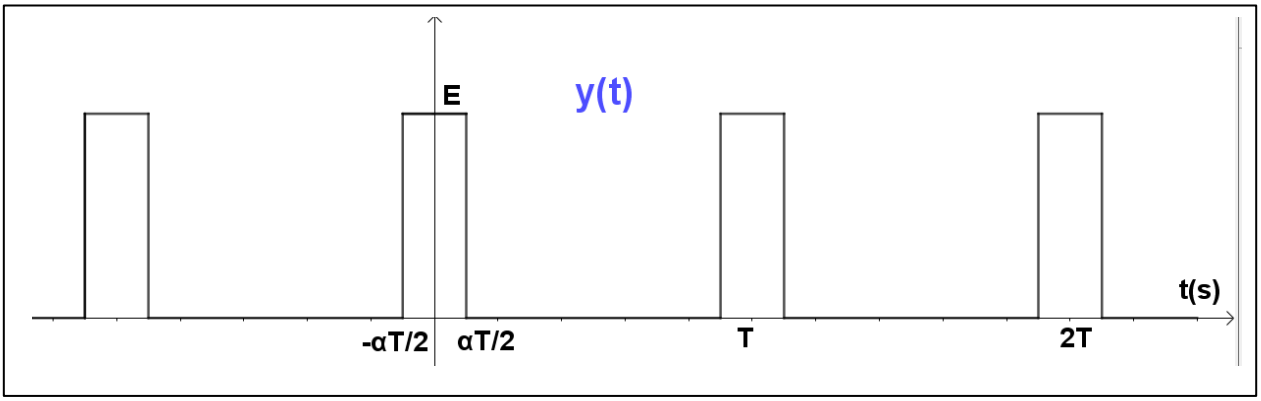


Figure 2: signal PWM

1) Calculer **théoriquement** la valeur moyenne et la valeur efficace de ce signal.

On admet que la décomposition en série de Fourier de ce signal est :

$$y(t) = \alpha E + \frac{2E}{\pi}\sin(\alpha\pi).\cos(wt) + \frac{2E}{2\pi}\sin(2\alpha\pi).\cos(2wt) + \frac{2E}{3\pi}\sin(3\alpha\pi).\cos(3wt) + ...$$

Soit en utilisant le signe somme : $y(t) = \alpha.E + \sum_{n=1}^{+\infty} \frac{2.E}{n.\pi}\sin(n.\alpha.\pi).\cos(n.\omega.t)$

Pour les applications numériques, on prendra $\boldsymbol{E = 5V}$; $\boldsymbol{\alpha = 0,2}$; $\boldsymbol{\omega = 2\pi.100.10^3\ rad/s}$.

III.) Étude du signal PWM : $y(t)$

III.1.a) Étude théorique du signal PWM

1.) $y_{moy} = \frac{1}{T}\int_0^T y(t)\,dt$

$= \frac{1}{T}\int_0^T \alpha E + \sum_{n=1}^{+\infty} \frac{2E}{n\pi}\sin(n\alpha\pi)\cdot\cos(n\omega t)$

$y(t) = E$ sur $[0; \alpha T]$

$y(t) = 0$ sur $[\alpha T; T]$

Donc :

$$y_{moy} = \frac{1}{T}\left(\int_0^{\alpha T} E\,dt + \int_{\alpha T}^{T} 0\,dt\right)$$

Ainsi y_{moy} sera de la valeur de : $\frac{1}{T}\int_0^{\alpha T} E\,dt$

$$y_{moy} = \frac{1}{T}\int_0^{\alpha T} E\,dt$$

$$= \frac{1}{T}[Et]_0^{\alpha T} = \frac{1}{T}[E(\alpha T) - E(0)]$$

Scanned with CamScanner

$$y_{moy} = \frac{1}{T}\cdot E(\alpha T) = \frac{E\alpha T}{T} = \boxed{E\alpha}$$

$$y_{eff} = \sqrt{\frac{1}{T}\int_0^T y^2(t)\,dt}$$

$$= \sqrt{\frac{1}{T}\int_0^{\alpha T} E^2\,dt}$$

$$= \sqrt{\frac{1}{T}[E^2 t]_0^{\alpha T}}$$

$$= \sqrt{\frac{1}{T}[E^2(\alpha T) - E^2(0)]}$$

$$= \sqrt{\frac{1}{T}E^2\alpha T}$$

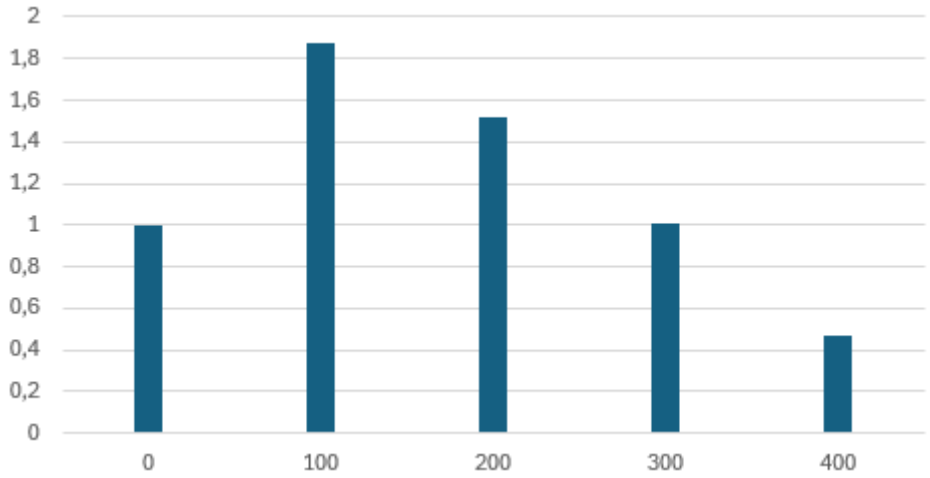
$$= \sqrt{E^2\alpha}$$

$$\boxed{y_{eff} = E\sqrt{\alpha}}$$

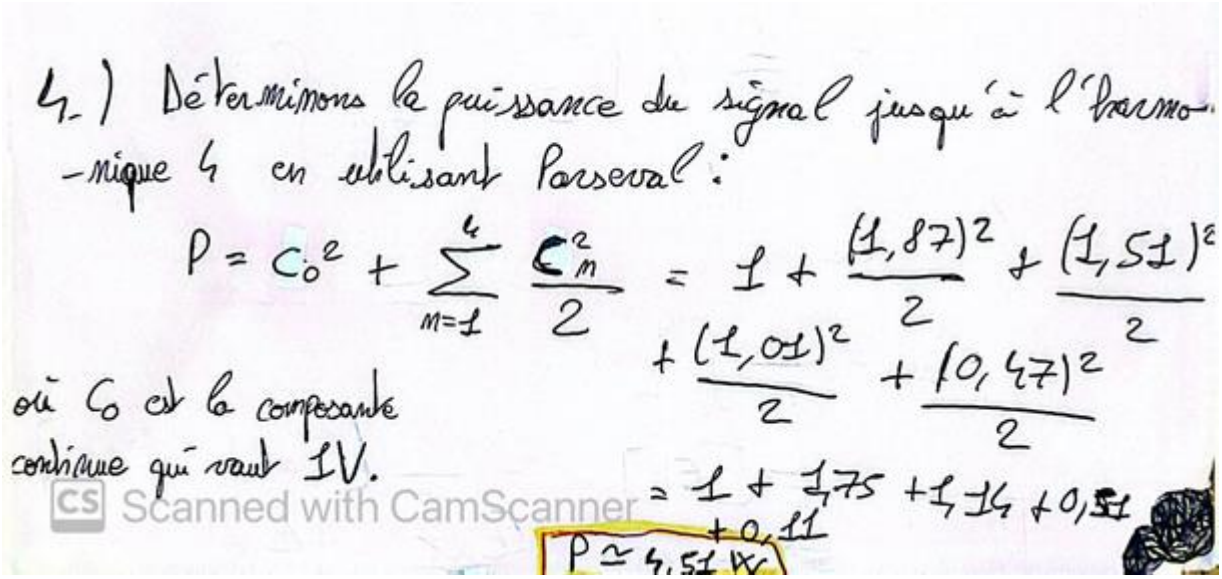
2) Calculer à l'aide d'un tableur la valeur numérique de la composante continue ainsi que l'amplitude du fondamental, et des harmoniques 2, 3 et 4.

n	f	A
0	0	1
1	100	1,87097857
2	200	1,51365346
3	300	1,0091023
4	400	0,46774464

3) Représenter avec le tableur le spectre en amplitude de ce signal jusqu'à $f = 400kHz$.



4) En utilisant Parseval, déterminer la puissance de ce signal (on se limitera à l'harmonique 4).



Sur excel

Parseval	4,51439005

III.1.b / Etude sous Matlab du signal PWM

- Réaliser un programme Matlab qui calcule la valeur moyenne, la valeur efficace et le spectre du signal y(t) importé de l'oscilloscope.

```
%% EXTRACTION DES HARMONIQUES ET THEOREME DE PARSEVAL
% Identification des pics avec une boucle
indices_pics = []; % On crée un tableau vide pour stocker la position des pics
for j = 2:(length(Amp_spectre) - 1)
    % Si le point est plus grand que son voisin de gauche ET de droite
    if (Amp_spectre(j) > Amp_spectre(j-1)) && (Amp_spectre(j) > Amp_spectre(j+1))
        % Ici on enregistre dans le tableau le numéro de l'échantillon
        indices_pics = [indices_pics, j];
    end
end

% Ajout de la composante continue (0 Hz) au tableau
% L'indice 1 correspond toujours à f = 0 Hz
tous_indices_candidats = [1, indices_pics];
% On prend les amplitudes correspondantes aux indices
toutes_amplitudes_candidates = Amp_spectre(tous_indices_candidats);

%% maxk
% On récupère les 4 plus grandes valeurs val_max parmi tous nos pics + Composante continue
(donc 5)
[val_max, idx_relatif] = maxk(toutes_amplitudes_candidates, 5);
indices_finaux = tous_indices_candidats(idx_relatif);
f_extraites = f(indices_finaux);
[freq_triees, ordre] = sort(f_extraites);
amp_triees = val_max(ordre);

for i = 1:length(freq_triees)
    fprintf('Harmonique n=%d | Freq: %7.0f Hz | Amplitude: %.3f V\n', i-1, freq_triees(i),
amp_triees(i));
end

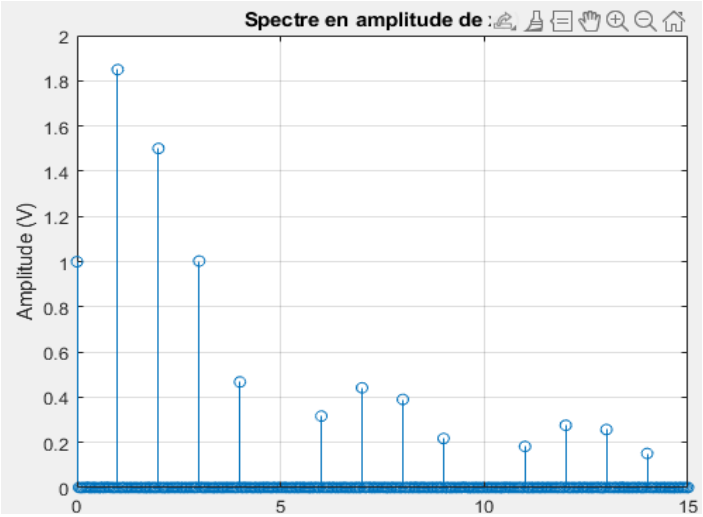
% Théorème de Parseval
% Puissance C_0 = (V_moy)^2
% Puissance C_N = Somme des (V_eff_harmoniques^2) -> équivalent à Somme( (V_max^2)/2 )
P_C0 = amp_triees(1)^2;
P_CN = sum((amp_triees(2:end).^2) / 2);
P_totale_Parseval = P_C0 + P_CN;
fprintf('Puissance totale calculée via Parseval : %.3f W\n', P_totale_Parseval);
```

Comparer les résultats avec ceux calculés dans la partie théorique.

On a les mêmes valeurs efficaces

x eff 2.2147

 Retrouver les amplitudes de la composante continue, du fondamental et des $N = 14$ premiers harmoniques (on pourra utiliser la fonction maxk).



 Appliquer le théorème de Parseval afin de déterminer la puissance de ce signal.

On a 4.447W ce qui est tres proche du signal trouver théoriquement.

P_totale_Parseval 4.4471

Partie 4 : Etude d'un canal Radio

Objectif : lors du TPn°3 des TP de Télécoms, vous avez acquis le spectre en réception d'une transmission radio. Cette transmission été effectuée autour d'une fréquence porteuse de f_p=27MHz.

On se propose d'analyser sous Matlab ce spectre et de la caractériser en effectuant les calculs suivants :

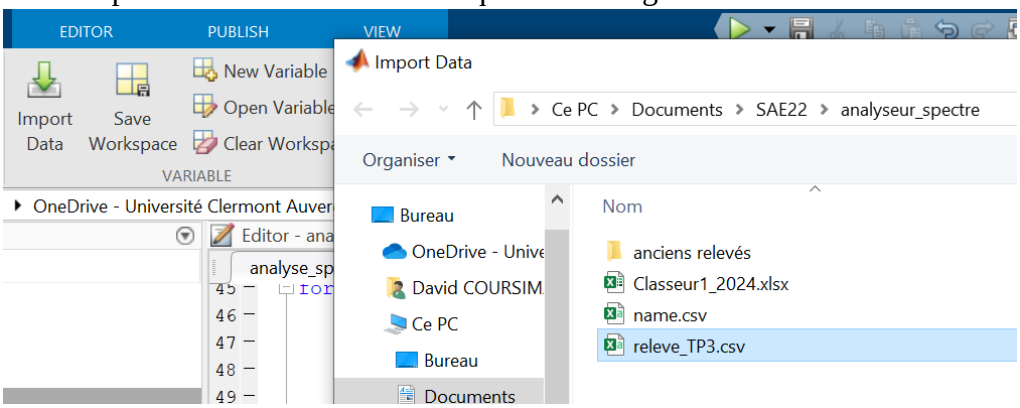
- Calcul de la bande occupée à 20dB
- Calcul de la densité spectrale de puissance (DSP)
- Détermination de la puissance totale du canal

I / Récupération des données

I.1 / Importation des données sur Matlab

Il faut importer les données sauvegardées pendant la séance de TP de Télécoms sur Matlab.

- Enregistrer le fichier dans le répertoire de votre projet Matlab.
- Importer le fichier comme indiqué dans la figure suivante :



- Sélectionner les données à importer puis choisir le type de sortie 'column vector'

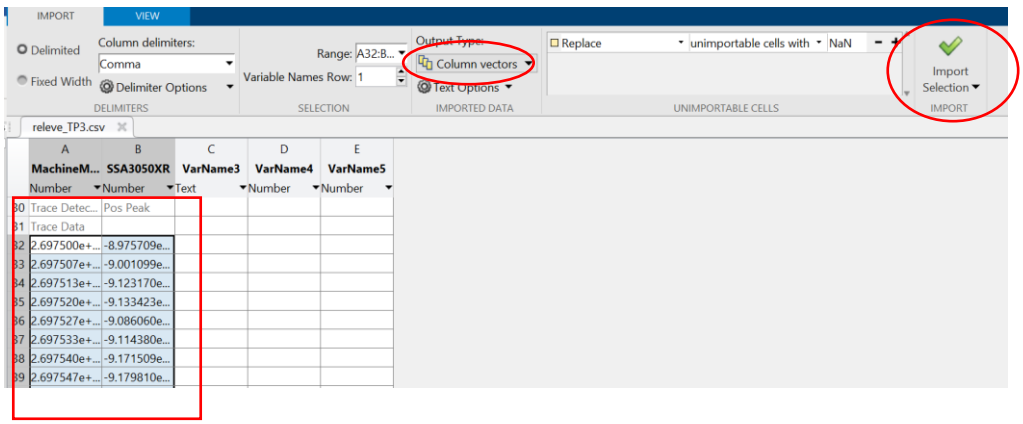


Figure 3: importation des données sur Matlab

- Importer ensuite les données
- Deux vecteurs colonnes ont dû être crées dans le Workspace. Cliquer sur ces vecteurs pour visualiser leurs contenus…

I.2 / Visualisation des données

✋ Créer un script permettant d'afficher l'évolution de la puissance en fonction de la fréquence.

```
plot(f, PL_dBm)
```

II / Lissage des données

Les données récupérées sont relativement bruitées. On va essayer de réduire ce bruit en appliquant un filtre appelé filtre de lissage ou filtre moyenneur. L'algorithme est le suivant :

Entrées :
P_{dBm} % tableau des puissances en dBm.
N_s % nombre de cases de ce tableau
K =4 % ordre du filtre (on fera la moyenne sur 2K+1 échantillons)

Initialisation :
Créer un tableau $P_{L\,dBm} = P_{dBm}$ % Ce tableau servira à stocker les données lissées.

Traitement :
Pour n allant de K+1 à N_s-K

$$P_{L\,dBm}(n) = \frac{\sum_{k=-K}^{K} P_{dBm}(n+k)}{2K+1}$$

fin

Sortie : $P_{L\,dBm}$

✋ Implanter cet algorithme et afficher le résultat.

```
FaireNs = length(PdBm); % Nombre total d'échantillons

K = 4;              % Ordre du filtre (moyenne sur 2K+1 points)

PL_dBm = PdBm;      % Création du tableau pour le signal lissé

% Boucle de calcul de la moyenne glissante

   for n = (K+1) : (Ns-K)

      PL_dBm(n) = sum(PdBm(n-K : n+K)) / (2*K + 1);

End
```

Faire varier l'ordre du filtre de lissage (paramètre K) et visualiser son effet
Si on augmente K la largeur de bande du spectre central augmente

III / Détermination de la bande occupée à $-20dB$

- En utilisant la fonction max, déterminer le niveau maximal de puissance dans ce relevé ainsi que la fréquence correspondante. Aide : la fonction « max » renvoie le maximum d'un tableau mais aussi l'indice correspondant…

```
[Pmax, ind_max] = max(PdBm);
% 2. Récupération de la fréquence correspondante grâce à l'indice
f_max = f(ind_max);
```

- Compléter le programme afin de déterminer la bande occupée de ce canal à $-20dB$.

```
Seuil = Pmax - 20;
indices_valides = find(PL_dBm >= Seuil);

% 3. Identifier les indices correspondant aux bornes de la bande (début et fin)
ind_inf = indices_valides(1);      % Premier indice (fréquence de coupure basse)
ind_sup = indices_valides(end);    % Dernier indice (fréquence de coupure haute)

% 4. Récupérer les fréquences correspondantes en utilisant ces indices
f_inf = f(ind_inf);
f_sup = f(ind_sup);

% 5. Calcul de la bande occupée à -20 dB
Bande_Occupee = f_sup - f_inf
```

IV / Calcul de la puissance du canal

- Convertir le tableau des puissances, PdBm, en un tableau exprimé en Watt : P_W.

```
P_W = 10.^((PdBm - 30)/10);
```

- En déduire un tableau des densités spectrales de puissances : $DSP = \frac{P_W}{RBW}$.

```
% Définir la résolution fréquentielle (RBW)
RBW = f(2) - f(1);
% Calcul de la DSP
DSP = P_W / RBW;
```

- Calculer la puissance du canal comme :

$$P_{canal} = \int_{f_{min}}^{f_{max}} DSP(f).df$$

```
%Calcul de la puissance du canal
df = f(2) - f(1);      % pas en fréquence
P_canal = sum(DSP) * df;
% Conversion en dBm (souvent demandé)
P_canal_dBm = 10*log10(P_canal) + 30;
Resultat :
```

P_canal 1.4212e-05

V / Présentation des résultats

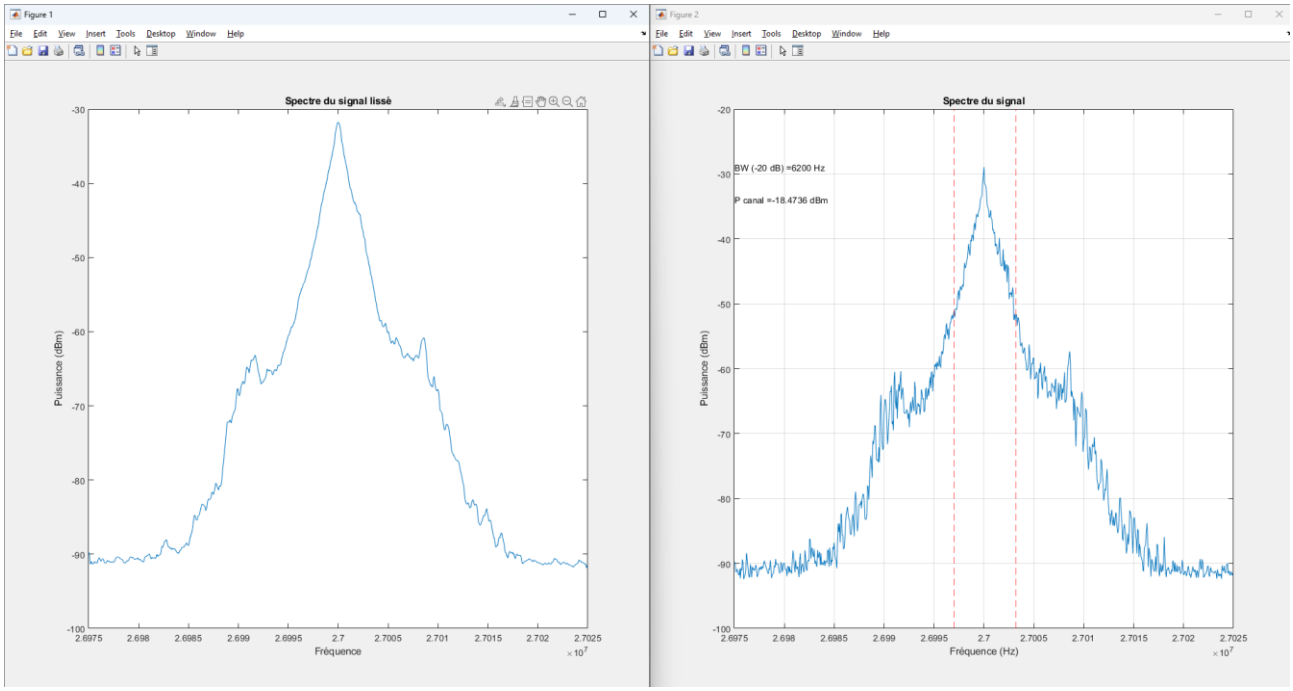
- Utiliser les commandes « strcat » et « text » afin d'afficher les calculs de la bande occupée à $20dB$ et de la puissance canal.

```
figure;
plot(f, PdBm); % CORRECTION : PdBm au lieu de P_dBm
grid on;
xlabel('Fréquence (Hz)');
ylabel('Puissance (dBm)');
title('Spectre du signal');

% Afficher la bande à -20 dB
hold on;
% Lignes verticales pour la bande
xline(f_inf, '--r');

xline(f_sup, '--r');
% Afficher les résultats avec text
% Texte bande passante
texte1 = strcat('BW (-20 dB) = ', num2str(Bande_occupee), ' Hz'); % CORRECTION :
Bande_occupee au lieu de BW
% Texte puissance canal
texte2 = strcat('P canal = ', num2str(P_canal_dBm), ' dBm');
% Placement du texte sur le graphe
text(f(1), max(PdBm), texte1);     % CORRECTION : PdBm
text(f(1), max(PdBm)-5, texte2);   % CORRECTION : PdBm;
```

Voici le résultat final :



On devra avoir un résultat analogue à celui de la Figure 4.

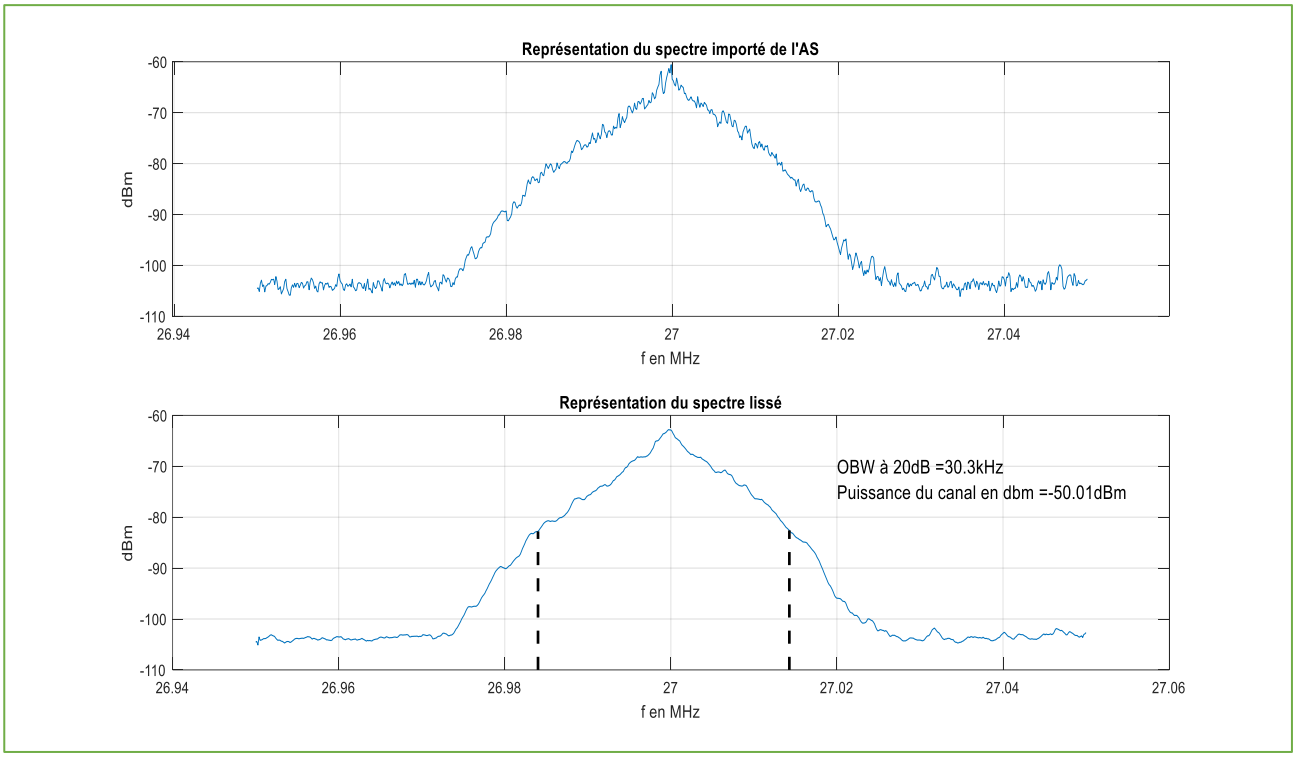


Figure 4: résultat final attendu